claude-windows / 05c32f87-3bfb-4a7f-861c-0050ac4169ac

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\05c32f87-3bfb-4a7f-861c-0050ac4169ac\subagents\workflows\wf_a8f43370-4cb\agent-a1c59d4e31029e95f.jsonl`
- SHA-256: `d5f59d8333553ad96ccda226b1740879d0d583cc6bbd957c8ad129ceb602f288`
- Source modified: `2026-07-07T15:22:48+00:00`
- Imported at: `2026-07-09T08:32:23+00:00`
- project: `wf_a8f43370-4cb`
- session_id: `05c32f87-3bfb-4a7f-861c-0050ac4169ac`

Transcript

1. user (2026-07-07T15:07:26.830Z)

```
You are reviewing a two-repo bug fix BEFORE it merges into a HOSTED PRODUCTION Cloud Run
control-plane.

THE BUG: On the hosted control-plane, POST /api/process with compute="local" ran the in-process
segmenter ON THE CONTROL-PLANE, which bakes NO detector weights (only the worker Cloud Run Job
GCSFuse-mounts weights at /gcs/models/...). It died deep with a cryptic
"WASB weights not found at /opt/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar".
[COMPUTE] requested=local actual=in_process dispatched=None status=failed.

THE FIX (2 parts):
- BACKEND (repo undefined, branch undefined, PR #undefined):
  backend/orchestration.py adds _WEIGHTS_BACKED_SEGMENTERS = {wasb_hybrid, tracknet_hybrid,
  fusion_hybrid} and _local_detector_weights_present(cfg, seg_name). In _maybe_dispatch_remote
the
  choice=="local" branch now fails fast via _failed(msg, False) when the segmenter is weights-
backed
  AND _cloud_available(cfg) AND local weights are not readable; otherwise returns None (in-
process as
  before). Plus 4 tests in tests/test_api_cloud_dispatch.py.
- SPA (repo undefined, branch undefined, PR #undefined):
  web/src/components/IngestPanel.tsx compute picker defaults to "cloud" when
  capabilities.deployment.compute_target==="cloud_run" (else keeps the free "local" default).
Plus 1 test.

AUTHORITATIVE CODE ? the working trees may NOT contain the change; read these snapshot files:
- undefined/backend.diff                    (backend commit diff)
- undefined/orchestration_new.py            (FULL post-change backend/orchestration.py)
- undefined/test_api_cloud_dispatch_new.py  (FULL post-change tests)
- undefined/spa.diff                        (SPA commit diff)
- undefined/IngestPanel_new.tsx             (FULL post-change IngestPanel.tsx)
You MAY read other repo files for ground truth, e.g. under undefined:
  backend/pipeline/detectors/native_wasb_runner.py, wasb_runner.py, tracknet_runner.py;
  backend/pipeline/segmenters/{wasb_hybrid,tracknet_hybrid,fusion_hybrid,trajectory_hybrid}.py;
  backend/config/models.py; backend/main.py (the /api/capabilities handler ~line 364-410);
and under undefined: web/src/api/types.ts, web/src/api/client.ts,
web/src/components/IngestPanel.test.tsx.
You may also run: gh pr diff undefined -R Khelsutra/badminton-highlight-indexer  and  gh pr diff
undefined -R avidullu/khelsutra.

Report ONLY real, concrete defects that would cause wrong behavior, a bad/incorrect merge, a
regression, or a materially worse UX ? each with a SPECIFIC failure scenario (concrete
config/inputs
```

? the wrong result). Verify claims against the actual code before reporting; do not invent style nits.
If nothing real exists in your dimension, return an empty findings array.

=== YOUR DIMENSION: conventions-and-regressions ===
REGRESSION risk + repo-convention fit. Does the change break any EXISTING test or behavior? (e.g. existing compute=local tests use segmenter="motion" ? confirm motion is unaffected; confirm no
existing weights-backed compute=local-on-cloud test now fails.) Does _failed's message style/tone
match the sibling reject at the choice=="cloud" branch? Is the frozenset / helper naming and comment
density consistent with the module? Is os.path.expanduser import/usage fine (os already imported)?
For the SPA: does defaulting to cloud interact correctly with the cost-ack gate (cloudBlocked) and the
sendCompute = cloudAvailable ? compute : undefined logic ? any state where the first submit is silently blocked or a paid job dispatches without ack? Any i18n/testid assumptions broken?

2. assistant (2026-07-07T15:07:32.843Z)

I'll start by locating the snapshot files and understanding the repo structure, since the prompt uses "undefined" placeholders for paths.

3. user (2026-07-07T15:07:35.850Z)

```
total 62
drwxr-xr-x 1 avidu 197609    0 Jul  5 12:57 .
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:57 ..
drwxr-xr-x 1 avidu 197609    0 Jun 22 11:12 .claude
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:33 .git
-rw-r--r-- 1 avidu 197609  297 Jun 20 23:27 .gitattributes
drwxr-xr-x 1 avidu 197609    0 Jun 20 12:43 .github
-rw-r--r-- 1 avidu 197609  323 Jun 22 14:57 .gitignore
drwxr-xr-x 1 avidu 197609    0 Jul  2 21:13 deploy
drwxr-xr-x 1 avidu 197609    0 Jul  2 21:13 docs
drwxr-xr-x 1 avidu 197609    0 Jun 21 23:55 locales
-rw-r--r-- 1 avidu 197609 9773 Jul  2 21:13 NEXT_STEPS.md
drwxr-xr-x 1 avidu 197609    0 Jun 21 15:38 node_modules
-rw-r--r-- 1 avidu 197609 1469 Jun 23 00:19 README.md
drwxr-xr-x 1 avidu 197609    0 Jun 23 00:19 site
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:45 web
---PARENT---
total 108
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:57 .
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:24 ..
drwxr-xr-x 1 avidu 197609    0 Jun 23 09:53 Annotation Setup
-rw-r--r-- 1 avidu 197609  905 Jul  2 21:20 AUDIT_REPORT_khelsutra-guru_2026-07-02.md
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:14 badminton-highlight-indexer
-rw-r--r-- 1 avidu 197609 1222 Jul  2 21:35 bash.exe.stackdump
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:34 bhi-wt-worker-reval
drwxr-xr-x 1 avidu 197609    0 Jul  5 12:57 khelsutra
drwxr-xr-x 1 avidu 197609    0 Jul  5 18:47 khelsutra-screencast
drwxr-xr-x 1 avidu 197609    0 Jul  5 13:44 marketing
drwxr-xr-x 1 avidu 197609    0 Jul  5 13:43 pr-158-screenshots
drwxr-xr-x 1 avidu 197609    0 Jun 22 15:03 rally-annotator
drwxr-xr-x 1 avidu 197609    0 Jul  4 06:58 sports-data-collector
drwxr-xr-x 1 avidu 197609    0 Jul  2 19:46 sports-obsreport
---FIND SNAPSHOTS---
```

4. assistant (2026-07-07T15:07:39.339Z)

Let me locate the snapshot files, which appear to be somewhere in the workspace.

5. user (2026-07-07T15:07:53.541Z)

C:\Users\avidu/AppData/Local/Temp/backend.diff
C:\Users\avidu/AppData/Local/Temp/spa.diff

6. assistant (2026-07-07T15:08:00.072Z)

Only the two diff files exist. Let me read them and locate the actual repos.

7. user (2026-07-07T15:08:00.473Z)

```
1       commit 4565c7436ccf0b4e30f68f01efe6082eff371a5d
2       Author: Avi Dullu <avi.dullu@gmail.com>
3       Date:   Tue Jul 7 19:56:22 2026 +0530
4
5           fix(serving): fail fast when compute=local forces weightless in-process detection
6
7           A hosted cloud-serving control-plane bakes NO detector weights into its image
8           (only the worker Cloud Run Job GCSFuse-mounts them at /gcs/models/...). So a
9           POST /api/process with compute="local" for a weights-backed trajectory detector
10          (wasb_hybrid / tracknet_hybrid / fusion_hybrid) returned None from
11          _maybe_dispatch_remote and ran the in-process segmenter ON THE CONTROL-PLANE,
12          which then died deep in the healthcheck with the confusing
13          "WASB weights not found at /opt/models/WASB-SBDT/..." (observed live 2026-07-07,
14          job f29d41d0?: [COMPUTE] requested=local actual=in_process ... status=failed).
15
16          Guardrail: on a cloud-serving-capable server (_cloud_available) whose local
17          weights for the requested detector aren't readable, a forced compute="local"
18          now fails FAST with an actionable message ("this hosted server can't run
19          detection locally ... Choose 'Run in cloud'.") ? mirroring the existing
20          compute="cloud"-not-available reject ? instead of the silent in-process trap.
21          A truly-local box, or a cloud-wired box that actually has the weights on disk,
22          is unaffected (the weights-present check is the escape hatch); a weight-free
23          segmenter (motion) is out of scope. Honest provenance: path=not_run,
24          requested=local, dispatched=False.
25
26          Covered by tests/test_api_cloud_dispatch.py (fail-fast, weights-present escape
27          hatch, motion unaffected, pure-local not guardrailed).
28
29          Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
30
31      diff --git a/backend/orchestration.py b/backend/orchestration.py
32      index 5e9f510..bfe39f5 100644
33      --- a/backend/orchestration.py
34      +++ b/backend/orchestration.py
35      @@ -155,6 +155,38 @@ def _ingest_remote_segments(
36           return n
37
38
39      +# Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS
box. A hosted
40      +# cloud-serving control-plane bakes NO detector weights into its image (only the worker
Cloud Run Job
41      +# GCSFuse-mounts them), so forcing compute="local" for one of these there can never
succeed ? the
42      +# guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic
deep
43      +# "WASB weights not found". (Names are the registered segmenter identities; see
backend/pipeline/segmenters.)
44      +_WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid",
"fusion_hybrid"})
45      +
46      +
47      +def _local_detector_weights_present(cfg, seg_name: str) -> bool:
48      +    """Whether the trajectory weights the IN-PROCESS ``seg_name`` detector needs are
```

```
present + readable
49      +    on THIS box. Mirrors the native/WSL runners' resolution (env override ?
``indexing.<det>.weights_path``)
50      +    and the ``/api/capabilities`` probe, so the guardrail agrees with the healthcheck
that would
51      +    otherwise fail deep in the run. Fusion follows its configured substrate (WASB
default / TrackNet).
52      +    A non-weights segmenter returns True (nothing local to check)."""
53      +    name = (seg_name or "").strip().lower()
54      +    if name not in _WEIGHTS_BACKED_SEGMENTERS:
55      +        return True
56      +    idx = getattr(cfg, "indexing", None)
57      +    substrate = (
58      +        getattr(getattr(idx, "fusion", None), "substrate", "") or "wasb"
59      +    ).strip().lower()
60      +    if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate ==
"tracknet"):
61      +        path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
62      +    else:  # wasb_hybrid, or fusion on the (default) WASB substrate
63      +        path = (
64      +            os.environ.get("WASB_WEIGHTS")
65      +            or getattr(getattr(idx, "wasb", None), "weights_path", "")
66      +            or ""
67      +        )
68      +    return bool(path) and os.path.exists(os.path.expanduser(path))
69      +
70      +
71       def _cloud_available(cfg) -> bool:
72           """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).
73
74      @@ -446,6 +478,23 @@ def _maybe_dispatch_remote(
75               )
76               choice = ""
77           if choice == "local":
78      +        # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
79      +        # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
80      +        # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
81      +        # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail
FAST with an
82      +        # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
83      +        # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it
runs in-process
84      +        # below exactly as before (the weights-present check is the escape hatch).
85      +        if (
86      +            (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
87      +            and _cloud_available(cfg)
88      +            and not _local_detector_weights_present(cfg, seg_name)
89      +        ):
90      +            return _failed(
91      +                f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
92      +                "live only on the cloud worker, not on the control-plane. Choose 'Run
in cloud'.",
93      +                False,
94      +            )
95               return None  # explicit "run on my machine" ? in-process regardless of the
server's target
96           if choice == "cloud":
97               if not _cloud_available(cfg):
98      diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
```

```
index 8382234..7993b7f 100644
--- a/tests/test_api_cloud_dispatch.py
+++ b/tests/test_api_cloud_dispatch.py
@@ -721,6 +721,114 @@ def
test_cloud_override_survives_api_boundary_on_local_server(tmp_path, monkeypa
         assert all(_meta(s)["source"] == "cloud_run" for s in segs)


+# --- hosted control-plane guardrail: compute='local' for a weights-backed detector
-------------
+# A hosted cloud-serving control-plane bakes NO detector weights (only the worker Cloud
Run Job mounts
+# them), so a forced compute='local' for wasb/tracknet/fusion can't run in-process
there. It must fail
+# FAST with an actionable "run in the cloud" message ? not fall through to the in-
process segmenter
+# that dies deep with a cryptic "WASB weights not found" (observed live: job f29d41d0?,
2026-07-07).
+
+
+def test_local_on_cloud_server_without_weights_fails_fast(cloud_env, monkeypatch):
+    """compute='local' + a weights-backed detector on a cloud_run server with NO local
weights ?
+    a clear pre-dispatch failure (never dispatched, never the cryptic deep 'weights not
found')."""
+    _db, _bucket, _mp = cloud_env
+    monkeypatch.delenv("WASB_WEIGHTS", raising=False)
+    m.global_config.indexing.wasb.weights_path =
"/nonexistent/wasb_badminton_best.pth.tar"
+    fake = _FakeCloudClient()
+    _inject_fake(monkeypatch, fake)
+
+    out = m._execute_processing(
+        m.ProcessRequest(
+            video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
+        )
+    )
+    assert out["status"] == "failed"
+    assert "can't run detection locally" in out["message"]
+    assert "Run in cloud" in out["message"]
+    assert "weights not found" not in out["message"]  # NOT the cryptic deep failure
+    assert fake.calls == []  # never dispatched
+    # Honest provenance: requested local, ran NOWHERE (pre-dispatch reject).
+    assert out["compute"]["requested"] == "local"
+    assert out["compute"]["dispatched"] is False
+    assert out["compute"]["path"] == "not_run"
+
+
+def test_local_on_cloud_server_with_weights_present_runs_in_process(
+    cloud_env, tmp_path, monkeypatch
+):
+    """Escape hatch: the guardrail must NOT over-block ? on a cloud-wired box that
ACTUALLY has the
+    detector weights on disk, compute='local' still runs the in-process segmenter (no
dispatch)."""
+    _db, _bucket, _mp = cloud_env
+    weights = tmp_path / "wasb_badminton_best.pth.tar"
+    weights.write_bytes(b"WEIGHTS")
+    monkeypatch.delenv("WASB_WEIGHTS", raising=False)
+    m.global_config.indexing.wasb.weights_path = str(weights)
+    monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
+    fake = _FakeCloudClient()
+    _inject_fake(monkeypatch, fake)
+
+    out = m._execute_processing(
```

```
153     +        m.ProcessRequest(
154     +            video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
155     +        )
156     +    )
157     +    assert out["status"] == "success"
158     +    assert fake.calls == []  # in-process, not dispatched
159     +    assert out["compute"]["path"] == "in_process"
160     +
161     +
162     +def test_local_non_weights_segmenter_on_cloud_server_still_runs_in_process(
163     +    cloud_env, monkeypatch
164     +):
165     +    """The guardrail is scoped to weights-backed detectors: a weight-free segmenter
(motion) with
166     +    compute='local' on a cloud server is unaffected ? it runs in-process, never fails
fast."""
167     +    _db, _bucket, _mp = cloud_env
168     +    m.global_config.indexing.wasb.weights_path =
"/nonexistent/should_not_matter.pth.tar"
169     +    monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
170     +    fake = _FakeCloudClient()
171     +    _inject_fake(monkeypatch, fake)
172     +
173     +    out = m._execute_processing(
174     +        m.ProcessRequest(
175     +            video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
176     +        )
177     +    )
178     +    assert out["status"] == "success"
179     +    assert fake.calls == []
180     +    assert out["compute"]["path"] == "in_process"
181     +
182     +
183     +def test_local_weights_backed_on_pure_local_server_is_not_guardrailed(
184     +    tmp_path, monkeypatch
185     +):
186     +    """The guardrail only fires on a cloud-capable server. On a pure-local server (no
cloud), a
187     +    weights-backed compute='local' request falls through to the in-process segmenter as
before ? the
188     +    'run in the cloud' message would be nonsense there (no cloud to fall back to)."""
189     +    db = Database(str(tmp_path / "t.db"))  # noqa: F841
190     +    monkeypatch.setattr(m, "db_instance", db)
191     +    cfg = AppConfig.model_validate({"deployment": {"compute_target": ""}})  # noqa:
F841
192     +    monkeypatch.setattr(m, "global_config", cfg)
193     +    monkeypatch.setattr(
194     +        m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
195     +    )
196     +    monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
197     +    monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
198     +    monkeypatch.delenv("WASB_WEIGHTS", raising=False)
199     +    cfg.indexing.wasb.weights_path = "/nonexistent/wasb.pth.tar"
200     +    monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
201     +    video = tmp_path / "v.mp4"
202     +    video.write_bytes(b"hashable-bytes")
203     +
204     +    out = m._execute_processing(
205     +        m.ProcessRequest(
206     +            video_path=str(video), segmenter="wasb_hybrid", compute="local"
207     +        )
208     +    )
209     +    # Not the hosted fast-fail ? it ran in-process (the local box owns its own weights
story).
210     +    assert out["status"] == "success"
```

```
211     +    assert out["compute"]["path"] == "in_process"
212     +
213     +
214      # --- compute-path PROVENANCE (the validation method: prove which backend ran)
------------------
215
216
217
```

8. user (2026-07-07T15:08:07.595Z)

```
1       commit 1470e2acdea4f2b478476150da61e844dce0a8e9
2       Author: Avi Dullu <avi.dullu@gmail.com>
3       Date:   Tue Jul 7 19:57:35 2026 +0530
4
5           fix(ingest): default the compute picker to CLOUD on a hosted control-plane
6
7           On a hosted cloud-serving control-plane (capabilities.deployment.compute_target
8           == "cloud_run") the engine bakes NO detector weights, so in-process LOCAL
9           detection always fails ("WASB weights not found"). The picker still defaulted to
10          LOCAL and sent compute="local", which the engine ran on the weightless
11          control-plane -> a confusing failure.
12
13          Default the picker to CLOUD when the server advertises compute_target=cloud_run
14          (cloud_available too), so the SPA never sends compute=local by default on the
15          hosted build. The free-LOCAL default is kept for a local appliance that merely
16          also offers cloud (compute_target != cloud_run). The cloud cost-acknowledgement
17          gate is unchanged (no paid dispatch without consent).
18
19          Pairs with the engine-side fast-fail guardrail (badminton-highlight-indexer
20          _maybe_dispatch_remote) for anyone who still explicitly picks LOCAL on the
21          hosted build. Covered by an IngestPanel test.
22
23          Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
24
25      diff --git a/web/src/components/IngestPanel.test.tsx
b/web/src/components/IngestPanel.test.tsx
26      index 685aa1b..b39e4f1 100644
27      --- a/web/src/components/IngestPanel.test.tsx
28      +++ b/web/src/components/IngestPanel.test.tsx
29      @@ -185,6 +185,30 @@ describe("IngestPanel", () => {
30           expect(bodies[0].compute).toBe("local");
31         });
32
33      +  it("defaults to CLOUD (never sends compute=local) on a hosted cloud_run control-
plane", async () => {
34      +    // A hosted control-plane bakes no detector weights, so LOCAL can't run there ? the
picker must
35      +    // default to CLOUD. The cost gate still applies (accept it, then the sole submit
is compute=cloud).
36      +    const bodies: Record<string, unknown>[] = [];
37      +    vi.stubGlobal(
38      +      "fetch",
39      +      fetchAll({
40      +        caps: { deployment: { cloud_available: true, compute_target: "cloud_run" } },
41      +        onProcessBody: (b) => bodies.push(b),
42      +        jobs: [{ job_id: "j", status: "done", video_id: "vidH", result: { status:
"success", video_id: "vidH" } }],
43      +      }),
44      +    );
45      +    render(<IngestPanel onLoaded={vi.fn()} />);
46      +    await screen.findByTestId("compute-picker");
47      +    // Defaulted to cloud ? the cost note + ack are shown up front (LOCAL would hide
them).
48      +    expect(screen.getByTestId("compute-cost")).toBeTruthy();
```

```
49      +    typeUri("gs://bucket/match.mp4");
50      +    fireEvent.click(screen.getByRole("checkbox")); // accept the cost gate
51      +    clickProcess();
52      +    await act(async () => {});
53      +    expect(bodies).toHaveLength(1);
54      +    expect(bodies[0].compute).toBe("cloud"); // never "local" by default on the hosted
build
55      +  });
56      +
57         it("shows a pasted host-path basename while processing and clears it on cancel",
async () => {
58           vi.useFakeTimers();
59           vi.stubGlobal(
60      diff --git a/web/src/components/IngestPanel.tsx b/web/src/components/IngestPanel.tsx
61      index 38ca3a9..edfd5d9 100644
62      --- a/web/src/components/IngestPanel.tsx
63      +++ b/web/src/components/IngestPanel.tsx
64      @@ -81,7 +81,9 @@ export function IngestPanel({ onLoaded, onBusyChange }: Props) {
65
66         // Compute-picker (PACKAGING compute-picker, item 3): if this server can dispatch to
a cloud GPU
67         // (capabilities.deployment.cloud_available), let the user choose where DETECT runs.
Default LOCAL
68      -  // (free, no surprise cost); CLOUD requires an explicit cost acknowledgement before
it can run.
69      +  // (free, no surprise cost) EXCEPT on a hosted cloud-serving control-plane
(compute_target=cloud_run):
70      +  // there the control-plane bakes no detector weights, so LOCAL can never run ?
default CLOUD instead
71      +  // (set from capabilities below), which requires an explicit cost acknowledgement
before it can run.
72         const [cloudAvailable, setCloudAvailable] = useState(false);
73         const [compute, setCompute] = useState<Compute>("local");
74         const [cloudAck, setCloudAck] = useState(false);
75      @@ -101,6 +103,13 @@ export function IngestPanel({ onLoaded, onBusyChange }: Props) {
76              .then((c) => {
77                if (!alive) return;
78                setCloudAvailable(c.deployment?.cloud_available === true);
79      +         // Hosted cloud-serving control-plane (compute_target=cloud_run) bakes NO
detector weights, so
80      +         // in-process LOCAL detection there always fails ("weights not found") ?
default the picker to
81      +         // CLOUD so the SPA never sends compute=local by default on the hosted build. A
local appliance
82      +         // that merely also offers cloud (compute_target?cloud_run) keeps the free
LOCAL default.
83      +         if (c.deployment?.cloud_available === true && c.deployment?.compute_target ===
"cloud_run") {
84      +           setCompute("cloud");
85      +         }
86                setBackends(c.storage?.backends ?? []);
87                setMaxUploadBytes(
88                  typeof c.max_upload_mb === "number" && c.max_upload_mb > 0 ? c.max_upload_mb
* 1024 * 1024 : undefined,
89
```

9. assistant (2026-07-07T15:08:11.779Z)

Now let me locate the actual repos and read the ground-truth files.

10. user (2026-07-07T15:08:16.630Z)

```
origin  https://github.com/avidullu/khelsutra.git (fetch)
origin  https://github.com/avidullu/khelsutra.git (push)
---WEB DIR---
```

```
a4.touch.test.tsx
a8.status.test.tsx
AnnotatePanel.test.tsx
AnnotatePanel.tsx
build-error.test.tsx
CompileResult.test.tsx
CompileResult.tsx
FfmpegBanner.test.tsx
FfmpegBanner.tsx
IngestPanel.test.tsx
---BHI backend---
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\orchestration.py
---BHI git---
origin  https://github.com/Khelsutra/badminton-highlight-indexer.git (fetch)
origin  https://github.com/Khelsutra/badminton-highlight-indexer.git (push)
d53851b perf(validators): decode scene_cut/blur/relevance frames in one shared pass
3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image
(#510)
66756ad chore(serving): forward WASB decode/tuning knobs to worker + release hardening (#509)
```

11. assistant (2026-07-07T15:08:52.500Z)

Both repos located. Backend at `badminton-highlight-indexer` (Khelsutra), SPA at `khelsutra` (avidullu). Let me check whether these commits are already in the repos and read the key ground-truth files.

12. user (2026-07-07T15:08:55.450Z)

```
=== BACKEND repo: is commit present? ===
4565c74 fix(serving): fail fast when compute=local forces weightless in-process detection
--- current branch/status ---
perf/validation-shared-decode-pass
 M backend/compute/__init__.py
 M backend/compute/base.py
 M backend/compute/cloud_run.py
?? score_golden_gate.py
?? score_local.py

=== SPA repo: is commit present? ===
1470e2a fix(ingest): default the compute picker to CLOUD on a hosted control-plane
--- current branch/status ---
fix/hosted-compute-picker-defaults-cloud
```

13. assistant (2026-07-07T15:09:07.288Z)

Both fix commits exist. Backend commit is `4565c74`; SPA commit `1470e2a` is checked out on `fix/hosted-compute-picker-defaults-cloud`. Let me extract the authoritative post-change backend files from the commit and read the SPA files from the working tree.

14. user (2026-07-07T15:09:13.630Z)

```
orchestration lines:
1145 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/orchestration_post.py
test lines:
1018 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/test_api_cloud_dispatch_post.py
--- is 4565c74 ancestor of current HEAD? ---
NO not ancestor
--- branches containing the fix ---
  fix/hosted-local-detection-guardrail
  remotes/origin/fix/hosted-local-detection-guardrail
```

15. user (2026-07-07T15:09:21.057Z)

```
"""Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of
#390).

    The full hash -> validate -> segment -> report pipeline (``_execute_processing``), the
    Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` /
``_ingest_remote_segments``),
    and the shared compile resolution + worker handler (``_resolve_compile`` /
``_execute_compile``).
    Extracted verbatim from ``backend.main`` to shrink the orchestrator; behavior is
unchanged.

    SEAM DISCIPLINE (same convention as ``backend.api.job_worker``): anything that must
remain
    monkeypatchable-on-``backend.main`` or that lives in ``backend.main`` (the transitional
    ``db_instance``/``global_config`` globals and the ``ProcessRequest`` model) is resolved
through
    the ``backend.main`` module object at call time (``import backend.main as _main``),
never as a
    bare local. ``backend.main`` re-exports every name below, so
``monkeypatch.setattr(backend.main,
    ...)`` and ``import backend.main as m; m._execute_processing(...)`` keep working
unchanged.
    This module must NOT import ``backend.main`` at module level (main imports us for the
re-export).
    Routed seams (rebind-patched on ``backend.main`` by tests): ``global_config``,
``db_instance``,
    ``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
    Object-attribute patches (``m.storage.fetch``, ``m.segmenter_registry.get``, ...) hit
the shared
    objects and need no routing.
    """

    import csv
    import json
    import logging
    import math
    import os
    import re
    import tempfile
    import threading
    from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
    from urllib.parse import quote

    from backend import storage
    from backend.config import AppConfig, StitchingConfig
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.stitcher.watermark_i18n import localize_watermark
    from backend.pipeline.validators.base import registry
    from backend.results import Failure
    from backend.utils.paths import output_base, safe_output_filename

    if (
        TYPE_CHECKING
    ):  # runtime import would be circular (main imports us for the re-export)
        from backend.main import ProcessRequest  # noqa: F401

    logger = logging.getLogger(__name__)

    # #466: serialize CPU/GPU-heavy LOCAL work ? in-process segmentation and the ffmpeg
compile stitch ?
    # so that raising the drain's worker count (to overlap I/O-bound REMOTE Cloud Run
dispatch/poll) never
    # accidentally parallelizes local work. A single self-hosted box serializes GPU work and
Gemini is
    # rate-fragile; a remote dispatch is a bucket-poll and never holds this lane, so N
```

```
remote jobs overlap
51      # while local/compile jobs stay 1-at-a-time. It's a plain mutex (no nesting) ? no
deadlock; when the
52      # drain is serial (max_workers=1) it's an uncontended no-op = today's behaviour.
53      _LOCAL_COMPUTE_LANE = threading.BoundedSemaphore(1)
54
55
56      def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
57          """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
58
59          On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
60          fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
61          keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
62          """
63          if segments:
64              db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
65          else:
66              db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
67
68
69      def _validation_failure_message(failures: List[Any]) -> str:
70          details = []
71          for failure in failures:
72              name = getattr(failure, "validator_name", None) or "validation"
73              message = (getattr(failure, "message", None) or "").strip()
74              details.append(f"{name}: {message}" if message else str(name))
75          if not details:
```

16. assistant (2026-07-07T15:09:25.142Z)

`os` is imported (line 25) ? so `os.path.expanduser`, `os.path.exists`, `os.environ.get` are all fine. Now let me read the dispatch logic and the `_failed`/`_cloud_available`/`choice=="cloud"` sibling reject.

17. user (2026-07-07T15:09:30.579Z)

```
=== defs / key markers with line numbers ===
163:_WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid", "fusion_hybrid"})
166:def _local_detector_weights_present(cfg, seg_name: str) -> bool:
173:    if name not in _WEIGHTS_BACKED_SEGMENTERS:
190:def _cloud_available(cfg) -> bool:
423:def _maybe_dispatch_remote(
445:    def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
451:        # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
461:                    "path": "cloud_run" if ran else "not_run",
480:    if choice == "local":
489:            (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
499:    if choice == "cloud":
701:def _execute_processing(
906:                    "path": compute_actual or "not_run",
912:            "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
934:            segmenter_requested=req.segmenter,
935:            segmenter_actual=segmenter_actual,
```

18. user (2026-07-07T15:09:35.568Z)

```
160     # GCSFuse-mounts them), so forcing compute="local" for one of these there can never
succeed ? the
161     # guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic
deep
162     # "WASB weights not found". (Names are the registered segmenter identities; see
```

```
backend/pipeline/segmenters.)
163     _WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid",
"fusion_hybrid"})
164
165
166     def _local_detector_weights_present(cfg, seg_name: str) -> bool:
167         """Whether the trajectory weights the IN-PROCESS ``seg_name`` detector needs are
present + readable
168         on THIS box. Mirrors the native/WSL runners' resolution (env override ?
``indexing.<det>.weights_path``)
169         and the ``/api/capabilities`` probe, so the guardrail agrees with the healthcheck
that would
170         otherwise fail deep in the run. Fusion follows its configured substrate (WASB
default / TrackNet).
171         A non-weights segmenter returns True (nothing local to check)."""
172         name = (seg_name or "").strip().lower()
173         if name not in _WEIGHTS_BACKED_SEGMENTERS:
174             return True
175         idx = getattr(cfg, "indexing", None)
176         substrate = (
177             getattr(getattr(idx, "fusion", None), "substrate", "") or "wasb"
178         ).strip().lower()
179         if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate ==
"tracknet"):
180             path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
181         else:  # wasb_hybrid, or fusion on the (default) WASB substrate
182             path = (
183                 os.environ.get("WASB_WEIGHTS")
184                 or getattr(getattr(idx, "wasb", None), "weights_path", "")
185                 or ""
186             )
187         return bool(path) and os.path.exists(os.path.expanduser(path))
188
189
190     def _cloud_available(cfg) -> bool:
191         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).
192
193         True when the control plane is wired for Cloud Run: either the configured default
target is
194         already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
195         (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
196         are present so a forced per-request override can actually dispatch + collect a
result.
197
198         Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work."""
199         deployment = getattr(cfg, "deployment", None)
200         if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
201             return True
202         has_job = bool(
203             os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
204         )
205         out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
206         return bool(has_job and out_root)
207
208
209     def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
210         """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
211         minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
comes
212         with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
"starting"
```

```
213         (queue/cold-start), so the very first tick already shows movement."""
214         minutes = int(elapsed_sec) // 60
215         if minutes < 1:
216             return f"segmenting ({seg_name}) on cloud GPU ? starting up"
217         return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
218
219
220     def submit_time_video_id(input_ref, cfg) -> Optional[str]:
221         """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
gs://
222         object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
keep
223         deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
no
224         ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
225         if getattr(input_ref, "backend", "") != "gcs":
226             return None
227         sc = _storage_settings(cfg)
228         if not sc:
229             return None
```

19. user (2026-07-07T15:09:42.472Z)

```
423     def _maybe_dispatch_remote(
424         cfg,
425         db,
426         req: "ProcessRequest",
427         video_id: str,
428         seg_name: str,
429         val_results,
430         play_wm: int,
431         cand_wm: int,
432         notify,
433     ) -> Optional[Tuple[Dict[str, Any], bool]]:
434         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
435         == "cloud_run"``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
436         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
437
438         Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
the cloud job
439         actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
observability report
440         doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
process segmenter
441         (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
non-``cloud_run`` target, so
442         the in-process path stays byte-identical)."""
443         from backend.compute import ComputeJobSpec, get_compute_backend
444
445         def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
446             # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
(id > play_wm);
447             # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
448             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
449             # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
only when the
450             # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
rc / ingest
451             # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
452             # intended backend, `requested` the picker choice, `dispatched` whether the
remote job ran.
453             return (
```

```
                {
                    "video_id": video_id,
                    "status": "failed",
                    "message": msg,
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                    "compute": {
                        "path": "cloud_run" if ran else "not_run",
                        "requested": req.compute,
                        "target": "cloud_run",
                        "dispatched": ran,
                    },
                },
                ran,
            )

        # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
        # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
        # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
        choice = (req.compute or "").strip().lower()
        if choice and choice not in ("local", "cloud"):
            logger.warning(
                "[API] ignoring unknown compute=%r (expected 'local'|'cloud'); using server
default.",
                req.compute,
            )
            choice = ""
        if choice == "local":
            # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
            # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
            # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
            # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail FAST
with an
            # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
            # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it runs
in-process
            # below exactly as before (the weights-present check is the escape hatch).
            if (
                (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
                and _cloud_available(cfg)
                and not _local_detector_weights_present(cfg, seg_name)
            ):
                return _failed(
                    f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
                    "live only on the cloud worker, not on the control-plane. Choose 'Run in
cloud'.",
                    False,
                )
            return None  # explicit "run on my machine" ? in-process regardless of the
server's target
        if choice == "cloud":
            if not _cloud_available(cfg):
                return _failed(
                    "cloud-serving was requested but this server isn't configured for it "
                    "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
                    False,
```

```
505                 )
506             compute = get_compute_backend(cfg, target_override="cloud_run")
507         else:
508             compute = get_compute_backend(
509                 cfg
510             )  # server default (default-OFF unless cloud_run)
511         if compute is None:
512             return None  # default-OFF ? run the in-process segmenter below (unchanged)
513 
514         storage_cfg = getattr(cfg, "storage", None)
515 
516         # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
517         try:
518             input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
519         except ValueError as e:
520             return _failed(
521                 f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
522             )
523         if input_ref.backend == "local":
524             # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
525             # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
526             # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
527             # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
528             # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
529             # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
530             # default, while a bare/local-looking root can't stage no matter what
input_backend says
531             # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
532             # gives us nothing to build the staged URI under).
533             input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
534             try:
535                 root_backend = (
536                     storage.parse_uri(input_root).backend if input_root else "local"
537                 )
538             except ValueError:  # unsupported scheme in input_root ? nowhere real to stage
to
539                 root_backend = "local"
540             if root_backend == "local":
541                 return _failed(
542                     "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
543                     "stage local uploads into; a path local to this machine can't be read by
the Cloud "
544                     "Run job.",
545                     False,
546                 )
547             local_src = input_ref.key  # resolved local path of the upload
548             staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
549             try:
550                 notify("staging upload to bucket")
551                 # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/
552                 # region/addressing for R2·DO·MinIO; GCS project/client) ? not a fresh
default client.
```

20. assistant (2026-07-07T15:09:57.416Z)

The backend guardrail logic is clear. Now the critical regression question: do any EXISTING tests do `compute="local"` with a weights-backed segmenter on a cloud server (which would now fail-fast instead of running in-process)? Let me examine the test file and fixtures.

21. user (2026-07-07T15:10:04.322Z)

```
=== all compute="local" usages ===
478:    """Server configured for cloud (compute_target=cloud_run), but a per-request
compute='local'
489:            video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
644:    cloud_run server, POST /api/process with compute='local' must persist (jobs.compute) +
reconstruct
724:# --- hosted control-plane guardrail: compute='local' for a weights-backed detector
-------------
726:# them), so a forced compute='local' for wasb/tracknet/fusion can't run in-process there. It
must fail
732:    """compute='local' + a weights-backed detector on a cloud_run server with NO local
weights ?
742:            video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
760:    detector weights on disk, compute='local' still runs the in-process segmenter (no
dispatch)."""
772:            video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
784:    compute='local' on a cloud server is unaffected ? it runs in-process, never fails
fast."""
793:            video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
805:    weights-backed compute='local' request falls through to the in-process segmenter as
before ? the
824:            video_path=str(video), segmenter="wasb_hybrid", compute="local"
858:    """On a cloud server, compute='local' runs in-process AND the result proves it:
path=in_process,
866:            video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
1004:        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="motion",
compute="local"),

=== segmenter= usages with wasb/tracknet/fusion ===
152:        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
171:        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
197:        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
214:        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
231:        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
247:        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
266:        m.ProcessRequest(video_path=local_path, segmenter="wasb_hybrid")
292:        m.ProcessRequest(video_path=local_path, segmenter="wasb_hybrid")
337:        m.ProcessRequest(video_path=local_path, segmenter="wasb_hybrid")
361:        m.ProcessRequest(video_path=str(tmp_path / "up.mp4"), segmenter="wasb_hybrid")
431:        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
445:            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
532:            video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="cloud"
563:            video_path=str(video), segmenter="wasb_hybrid", compute="cloud"
581:                video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="banana"
742:            video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
772:            video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
824:            video_path=str(video), segmenter="wasb_hybrid", compute="local"
844:            video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="cloud"
918:            video_path=str(video), segmenter="wasb_hybrid", compute="cloud"

=== fixtures/helpers defs ===
30:def _pass():
39:class _FakeCloudClient(CloudRunJobClient):
98:def cloud_env(tmp_path, monkeypatch):
127:def _inject_fake(monkeypatch, fake):
141:def _meta(row):
146:def test_cloud_dispatch_ingests_segments(cloud_env):
147:    db, _bucket, monkeypatch = cloud_env
165:def test_cloud_dispatch_builds_correct_spec(cloud_env):
```

```
166:    _db, bucket, monkeypatch = cloud_env
189:def test_cloud_dispatch_forwards_skip_ai_handoff_true(cloud_env):
192:    _db, _bucket, monkeypatch = cloud_env
202:def test_cloud_dispatch_forwards_wasb_serving_knobs(cloud_env):
207:    _db, _bucket, monkeypatch = cloud_env
222:def test_cloud_dispatch_omits_unset_wasb_tuning(cloud_env):
226:    _db, _bucket, monkeypatch = cloud_env
239:def test_cloud_failure_marks_failed_no_rows(cloud_env):
240:    db, _bucket, monkeypatch = cloud_env
259:def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
260:    _db, _bucket, monkeypatch = cloud_env
273:def test_cloud_stages_local_upload_to_bucket(cloud_env, tmp_path, monkeypatch):
277:    db, _bucket, _mp = cloud_env
315:    cloud_env, tmp_path, monkeypatch
321:    _db, _bucket, _mp = cloud_env
351:def test_cloud_local_upload_without_input_root_still_fails(cloud_env, tmp_path,
monkeypatch):
354:    _db, _bucket, _mp = cloud_env  # fixture leaves input_root unset (input_backend=local)
384:    class _InProcSeg:
409:def test_cloud_ingest_failure_marks_failed_no_leak(cloud_env):
412:    db, _bucket, monkeypatch = cloud_env
438:def test_cloud_video_id_divergence_is_tolerated_with_warning(cloud_env, caplog):
441:    db, _bucket, monkeypatch = cloud_env
455:class _InProcSeg:
477:def test_per_request_local_forces_in_process(cloud_env):
480:    db, _bucket, monkeypatch = cloud_env
570:def test_unknown_compute_value_falls_back_to_server_default(cloud_env, caplog):
573:    db, _bucket, monkeypatch = cloud_env
589:def test_capabilities_reports_cloud_available(cloud_env):
592:    _db, _bucket, _monkeypatch = cloud_env
614:def test_full_api_process_to_query_via_cloud(cloud_env):
617:    db, _bucket, monkeypatch = cloud_env
642:def test_local_override_survives_api_boundary(cloud_env):
```

22. assistant (2026-07-07T15:10:17.712Z)

Line 477 `test_per_request_local_forces_in_process` uses `motion` ? safe. The key regression risk is `test_local_override_survives_api_boundary` (line 642) which does `compute="local"`. Let me check its segmenter and weights setup.

23. user (2026-07-07T15:10:20.686Z)

```
455     class _InProcSeg:
456         """Minimal in-process segmenter (writes one play row) ? proves the local path
actually ran."""
457
458         def __init__(self, db_, cfg_):
459             self.db = db_
460
461         def process_video(self, vid, path, *a, **k):
462             sid = self.db.add_play_segment(vid, 0.0, 5.0)
463             return [
464                 {
465                     "id": sid,
466                     "start_time": 0.0,
467                     "end_time": 5.0,
468                     "duration": 5.0,
469                     "metadata": {},
470                 }
471             ]
472
473
474     # --- item 3: SPA compute-picker (per-request compute override + cloud_available
capability) ---
475
```

```
476
477     def test_per_request_local_forces_in_process(cloud_env):
478         """Server configured for cloud (compute_target=cloud_run), but a per-request
compute='local'
479         forces the in-process segmenter and NEVER dispatches to Cloud Run."""
480         db, _bucket, monkeypatch = cloud_env
481         fake = _FakeCloudClient()
482         _inject_fake(monkeypatch, fake)
483         monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
484
485         out = m._execute_processing(
486             m.global_config,
487             m.db_instance,
488             m.ProcessRequest(
489                 video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
490             ),
491         )
492         assert out["status"] == "success"
493         assert len(out["play_segments"]) == 1
494         assert "cloud_run" not in out["message"]
495         assert (
496             fake.calls == []
497         )  # explicit "run on my machine" ? no dispatch despite a cloud server
498
499
500     def test_per_request_cloud_forces_dispatch_on_local_server(tmp_path, monkeypatch):
501         """Server default is LOCAL (compute_target=''), but a per-request compute='cloud'
forces a Cloud
502         Run dispatch when the control plane is wired (CLOUD_RUN_JOB + GOOGLE_CLOUD_PROJECT +
a bucket)."""
503         db = Database(str(tmp_path / "t.db"))  # noqa: F841
504         monkeypatch.setattr(m, "db_instance", db)
505         bucket = tmp_path / "bucket"
506         bucket.mkdir()
507         cfg = AppConfig.model_validate(
508             {  # noqa: F841
509                 "deployment": {"compute_target": ""},  # server default = in-process
510                 "storage": {"backend": "local", "output_root": str(bucket)},
511             }
512         )
513         monkeypatch.setattr(m, "global_config", cfg)
514         monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
515         fake_local = tmp_path / "fetched.mp4"
516         fake_local.write_bytes(b"FAKEVIDEOBYTES")
517         monkeypatch.setattr(
518             m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
519         )
520         monkeypatch.setattr(
521             m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
522         )
523         monkeypatch.setenv("CLOUD_RUN_JOB", "rally-worker")
524         monkeypatch.setenv("GOOGLE_CLOUD_PROJECT", "proj")
525         fake = _FakeCloudClient()
526         _inject_fake(monkeypatch, fake)
527
528         out = m._execute_processing(
529             m.global_config,
530             m.db_instance,
531             m.ProcessRequest(
532                 video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="cloud"
533             ),
534         )
535         assert out["status"] == "success"
536         assert len(out["play_segments"]) == 2
537         assert (
```

```
538             len(fake.calls) == 1
539         )  # per-request override dispatched despite the LOCAL server default
540
541
542     def test_per_request_cloud_unconfigured_fails_clearly(tmp_path, monkeypatch):
543         """compute='cloud' on a server NOT wired for it fails clearly (no silent local
fallback, no
544         dispatch) so the picker can surface 'cloud isn't available here'."""
545         db = Database(str(tmp_path / "t.db"))  # noqa: F841
546         monkeypatch.setattr(m, "db_instance", db)
547         cfg = AppConfig.model_validate(
548             {"deployment": {"compute_target": ""}}
549         )  # no bucket, no env  # noqa: F841
550         monkeypatch.setattr(m, "global_config", cfg)
551         monkeypatch.setattr(
552             m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
553         )
554         monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
555         monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
556         video = tmp_path / "v.mp4"
557         video.write_bytes(b"hashable-bytes")
558
559         out = m._execute_processing(
560             m.global_config,
561             m.db_instance,
562             m.ProcessRequest(
563                 video_path=str(video), segmenter="wasb_hybrid", compute="cloud"
564             ),
565         )
566         assert out["status"] == "failed"
567         assert "isn't configured" in out["message"]
568
569
570     def test_unknown_compute_value_falls_back_to_server_default(cloud_env, caplog):
571         """An unrecognised compute value is ignored (logged) and the server default applies
? here a
572         cloud_run server still dispatches to the cloud."""
573         db, _bucket, monkeypatch = cloud_env
574         fake = _FakeCloudClient()
575         _inject_fake(monkeypatch, fake)
576         with caplog.at_level("WARNING", logger="backend.main"):
577             out = m._execute_processing(
578                 m.global_config,
579                 m.db_instance,
580                 m.ProcessRequest(
581                     video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="banana"
582                 ),
583             )
584         assert out["status"] == "success"
585         assert len(fake.calls) == 1  # fell back to the configured cloud_run default
586         assert any("unknown compute" in r.getMessage() for r in caplog.records)
587
588
589     def test_capabilities_reports_cloud_available(cloud_env):
590         """A cloud_run-configured server advertises deployment.cloud_available=True (the SPA
shows the
591         cloud toggle only then)."""
592         _db, _bucket, _monkeypatch = cloud_env
593         app = m.create_app(m.global_config, m.db_instance)
594         client = TestClient(app)
```

24. assistant (2026-07-07T15:10:30.710Z)

I notice a critical discrepancy: the NEW tests call
`m._execute_processing(m.ProcessRequest(...))` with ONE arg, but EXISTING tests call

`m._execute_processing(m.global_config, m.db_instance, m.ProcessRequest(...))` with THREE args.
Let me check the actual signature of `_execute_processing`.

25. user (2026-07-07T15:10:33.660Z)

```
701     def _execute_processing(
702         config=None, db=None, req=None, progress=None
703     ) -> Dict[str, Any]:
704         """The full hash ? validate ? segment ? report pipeline for one video.
705
706         Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
707         module-level ``global_config``/``db_instance`` (backward-compatible with existing
708         tests that pass only ``req``). Route handlers always pass explicit params; older
709         test code that calls ``_execute_processing(req)`` still works.
710
711         ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
712         Raises on unexpected internal errors ? the caller records those as a job failure
713         """
714         import backend.main as _main  # call-time seam: main's globals/model stay patchable
715
716         # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
717         # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
718         if config is not None and not isinstance(config, AppConfig):
719             # Old signature: _execute_processing(req, progress=None)
720             if req is None and db is not None and isinstance(db, _main.ProcessRequest):
721                 req = db
722                 db = None
723             elif req is None and isinstance(config, _main.ProcessRequest):
724                 req = config
725                 config = None
726         # Resolve config/db from module globals when not passed explicitly
727         if config is None:
728             config = _main.global_config
729         if db is None:
730             db = _main.db_instance
731
732         notify = progress or (lambda stage: None)
733
734         notify("hashing")
735         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
736         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
737         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
738         # consumers (hash / validators / segmenter) use the resolved local path.
739         local_video = ""
740         try:
```

26. assistant (2026-07-07T15:10:43.854Z)

The shim handles single-arg calls (config=req ? shifted to req). Not a bug. Now let me read
`test_local_override_survives_api_boundary` (line 642) and
`test_full_api_process_to_query_via_cloud` ? these use compute='local' and could regress.

27. user (2026-07-07T15:10:44.054Z)

```
614     def test_full_api_process_to_query_via_cloud(cloud_env):
615         """End-to-end through the public API: POST /api/process (gs:// input) ? the inline
worker dispatches
616         to (fake) Cloud Run ? job done ? POST /api/query returns the ingested segments."""
617         db, _bucket, monkeypatch = cloud_env
```

```
        fake = _FakeCloudClient(video_id="vidCloud2")
        _inject_fake(monkeypatch, fake)
        app = m.create_app(m.global_config, db)
        client = TestClient(app)

        r = client.post(
            "/api/process",
            json={"video_path": "gs://b/clip.mp4", "segmenter": "wasb_hybrid"},
        )
        assert r.status_code == 202
        job_id = r.json()["job_id"]

        job = client.get(f"/api/jobs/{job_id}").json()
        assert job["status"] == "done"
        video_id = job["video_id"]
        assert video_id

        q = client.post("/api/query", json={"video_id": video_id})
        assert q.status_code == 200
        segs = q.json()["play_segments"]
        assert len(segs) == 2
        assert all(_meta(s)["source"] == "cloud_run" for s in segs)


    def test_local_override_survives_api_boundary(cloud_env):
        """REGRESSION: the picker choice must survive the async submit?worker?reconstruct
boundary. On a
        cloud_run server, POST /api/process with compute='local' must persist (jobs.compute)
+ reconstruct
        (_job_to_request) the choice and run IN-PROCESS ? not dispatch to Cloud Run. (An
earlier cut
        dropped compute in create_job/_job_to_request, silently nullifying the picker
through /api/process.)"""
        db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
        app = m.create_app(m.global_config, db)
        client = TestClient(app)

        r = client.post(
            "/api/process",
            json={
                "video_path": "gs://b/clip.mp4",
                "segmenter": "motion",
                "compute": "local",
            },
        )
        assert r.status_code == 202
        job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
        assert job["status"] == "done"
        assert (
            fake.calls == []
        )  # 'local' honoured across the boundary ? never dispatched to Cloud Run
        segs = db.query_play_segments(job["video_id"], {})
        assert len(segs) == 1
        assert all(
            _meta(s).get("source") != "cloud_run" for s in segs
        )  # in-process rows, not cloud


    def test_cloud_override_survives_api_boundary_on_local_server(tmp_path, monkeypatch):
        """REGRESSION (mirror): on a LOCAL-default server wired for cloud (env + bucket),
POST /api/process
        with compute='cloud' must survive the boundary and dispatch to Cloud Run."""
```

```
678         db = Database(str(tmp_path / "t.db"))  # noqa: F841
679         monkeypatch.setattr(m, "db_instance", db)
680         bucket = tmp_path / "bucket"
681         bucket.mkdir()
682         cfg = AppConfig.model_validate(
683             {  # noqa: F841
684                 "deployment": {"compute_target": ""},  # server default = in-process
685                 "storage": {"backend": "local", "output_root": str(bucket)},
686             }
687         )
688         monkeypatch.setattr(m, "global_config", cfg)
689         monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
690         fake_local = tmp_path / "fetched.mp4"
691         fake_local.write_bytes(b"FAKEVIDEOBYTES")
692         monkeypatch.setattr(
693             m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
694         )
695         monkeypatch.setattr(
696             m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
697         )
698         monkeypatch.setenv("CLOUD_RUN_JOB", "rally-worker")
699         monkeypatch.setenv("GOOGLE_CLOUD_PROJECT", "proj")
700         fake = _FakeCloudClient(video_id="vidBoundary")
701         _inject_fake(monkeypatch, fake)
702         app = m.create_app(m.global_config, db)
703         client = TestClient(app)
704 
705         r = client.post(
706             "/api/process",
707             json={
708                 "video_path": "gs://b/clip.mp4",
709                 "segmenter": "wasb_hybrid",
710                 "compute": "cloud",
711             },
712         )
713         assert r.status_code == 202
714         job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
715         assert job["status"] == "done"
716         assert (
717             len(fake.calls) == 1
718         )  # 'cloud' honoured across the boundary ? dispatched despite local default
719         segs = db.query_play_segments(job["video_id"], {})
720         assert len(segs) == 2
721         assert all(_meta(s)["source"] == "cloud_run" for s in segs)
722 
723 
```

28. assistant (2026-07-07T15:11:22.226Z)

All existing `compute="local"` tests use `motion` (lines 489, 659, 866, 1004) ? the guardrail is scoped to weights-backed segmenters, so none regress. Now the SPA, where subtle state bugs live. Let me read the full IngestPanel.tsx and its cost-ack/sendCompute logic.

29. user (2026-07-07T15:11:23.502Z)

```
1       // The front-half ingest screen (appliance S1/S2; tracker B-P2 + T4). Two ways to point
the tool at a
2       // video: UPLOAD a local file (primary ? alpha users have a file, not a URI; chunked to
the engine's
3       // /api/upload/* endpoints) or paste a `gs://` bucket URI / host path (secondary). Both
converge on
4       // the SAME back-half: POST /api/process ? poll GET /api/jobs/{id} ? hand the video_id
to App. Honest
5       // by construction: the engine reports coarse phase WORDS (not a %), and a success-
shaped failure
```

```
      // (done but no rallies) is shown, never silently treated as a load.
      import { useEffect, useState } from "react";
      import type { ChangeEvent, FormEvent } from "react";
      import { Trans, useTranslation } from "react-i18next";
      import { createAsset, getCapabilities } from "../api/client";
      import type { StorageBackendInfo } from "../api/types";
      import { useIngestJob } from "../hooks/useIngestJob";
      import { useUpload } from "../hooks/useUpload";
      import { classifyError, type IngestError } from "../lib/errors";
      import { errorText } from "../lib/errorText";
      import { log, newCorrelationId } from "../lib/log";
      import { StatusChip, Spinner } from "./StatusChip";

      type Compute = "local" | "cloud";
      // The three storage-medium choices the picker offers. "bucket" covers s3 OR gcs (the
user pastes a
      // full scheme'd URI); "drive" is a mounted Google Drive folder
(STUDIO_MEDIA_LIFECYCLE.md P5, D14).
      type Medium = "local" | "bucket" | "drive";

      interface Props {
        /** Called with the engine's video_id once a submitted job finishes successfully. */
        onLoaded: (videoId: string, displayName?: string) => void;
        /** Lets the shell hide demo-only controls while a real ingest is in flight. */
        onBusyChange?: (busy: boolean) => void;
      }

      function formatElapsed(totalSec: number): string {
        const sec = Math.max(0, Math.floor(totalSec));
        const mm = Math.floor(sec / 60);
        const ss = sec % 60;
        return `${mm}:${String(ss).padStart(2, "0")}`;
      }

      function formatEta(totalSec: number | undefined): string | null {
        if (typeof totalSec !== "number" || !Number.isFinite(totalSec) || totalSec < 0) return
null;
        const sec = Math.max(0, Math.ceil(totalSec));
        const hh = Math.floor(sec / 3600);
        const mm = Math.floor((sec % 3600) / 60);
        const ss = sec % 60;
        if (hh > 0) return `${hh}:${String(mm).padStart(2, "0")}:${String(ss).padStart(2,
"0")}`;
        return `${mm}:${String(ss).padStart(2, "0")}`;
      }

      function formatUploadSize(bytes: number): string {
        const b = Math.max(0, Number.isFinite(bytes) ? bytes : 0);
        if (b === 0) return "0 MB";
        const gb = b / 1024 ** 3;
        if (gb >= 1) {
          const rounded = Math.round(gb * 10) / 10;
          return `${Number.isInteger(rounded) ? rounded : rounded.toFixed(1)} GB`;
        }
        const mb = b / 1024 ** 2;
        if (mb >= 1) return `${Math.max(1, Math.round(mb))} MB`;
        return `${Math.max(1, Math.round(b / 1024))} KB`;
      }

      function sourceLabel(value: string): string | null {
        const trimmed = value.trim();
        if (!trimmed) return null;
        try {
          const parsed = new URL(trimmed);
          const tail = parsed.pathname.split(/[\\/]/).filter(Boolean).pop();
```

```
67          return tail ? decodeURIComponent(tail) : trimmed;
68        } catch {
69          return trimmed.split(/[\\/]/).filter(Boolean).pop() ?? trimmed;
70        }
71      }
72
73      export function IngestPanel({ onLoaded, onBusyChange }: Props) {
74        const { t, i18n } = useTranslation();
75        const [uri, setUri] = useState("");
76        const [activeSource, setActiveSource] = useState<string | null>(null);
77        const [selectedFile, setSelectedFile] = useState<File | null>(null);
78        const { phase, submit, cancel: cancelIngest, reset: resetIngest, busy: ingestBusy } =
useIngestJob(onLoaded);
79        // The UI locale recorded with the submission (PMF analytics + the localized reel
watermark).
80        const locale = i18n.resolvedLanguage ?? i18n.language;
81
82        // Compute-picker (PACKAGING compute-picker, item 3): if this server can dispatch to a
cloud GPU
83        // (capabilities.deployment.cloud_available), let the user choose where DETECT runs.
Default LOCAL
84        // (free, no surprise cost) EXCEPT on a hosted cloud-serving control-plane
(compute_target=cloud_run):
85        // there the control-plane bakes no detector weights, so LOCAL can never run ? default
CLOUD instead
86        // (set from capabilities below), which requires an explicit cost acknowledgement
before it can run.
87        const [cloudAvailable, setCloudAvailable] = useState(false);
88        const [compute, setCompute] = useState<Compute>("local");
89        const [cloudAck, setCloudAck] = useState(false);
90        // Storage-medium picker (P5): which media THIS box can store into
(capabilities.storage.backends,
91        // usable-only). Absent/local-only ? no picker (today's flow unchanged).
92        const [backends, setBackends] = useState<StorageBackendInfo[]>([]);
93        const [medium, setMedium] = useState<Medium>("local");
94        const [mediumUri, setMediumUri] = useState("");
95        const [storing, setStoring] = useState(false);
96        const [storeError, setStoreError] = useState<IngestError | null>(null);
97        // The engine's max upload size (BYTES), so an oversized file fails INSTANTLY client-
side instead of
98        // uploading into a 413. Absent ? undefined ? no guard (the engine still fail-fasts).
99        const [maxUploadBytes, setMaxUploadBytes] = useState<number | undefined>(undefined);
100       useEffect(() => {
101         let alive = true;
102         getCapabilities()
103           .then((c) => {
104             if (!alive) return;
105             setCloudAvailable(c.deployment?.cloud_available === true);
106             // Hosted cloud-serving control-plane (compute_target=cloud_run) bakes NO
detector weights, so
107             // in-process LOCAL detection there always fails ("weights not found") ? default
the picker to
108             // CLOUD so the SPA never sends compute=local by default on the hosted build. A
local appliance
109             // that merely also offers cloud (compute_target?cloud_run) keeps the free LOCAL
default.
110             if (c.deployment?.cloud_available === true && c.deployment?.compute_target ===
"cloud_run") {
111               setCompute("cloud");
112             }
113             setBackends(c.storage?.backends ?? []);
114             setMaxUploadBytes(
115               typeof c.max_upload_mb === "number" && c.max_upload_mb > 0 ? c.max_upload_mb *
1024 * 1024 : undefined,
116             );
```

```
          })
          .catch((err) => log("warn", "capabilities.unavailable", { error: String(err) }));
        return () => {
          alive = false;
        };
      }, []);
      // Only send a choice when the picker is actually shown; otherwise omit it (? server
default).
      const sendCompute = cloudAvailable ? compute : undefined;
      // CLOUD selected but the cost note not yet accepted ? block submit (don't silently
run local, and
      // don't dispatch a paid job without consent).
      const cloudBlocked = cloudAvailable && compute === "cloud" && !cloudAck;

      // Which medium segments to offer: local always; "bucket" if s3 OR gcs is usable;
"drive" if a Drive
      // mount is usable. The picker only appears when there's a REAL choice (a non-local
usable backend).
      const canUse = (id: string) => backends.some((b) => b.id === id && b.usable);
      const canBucket = canUse("s3") || canUse("gcs");
      const canDrive = canUse("gdrive");
      const mediumOptions: Medium[] = ["local", ...(canBucket ? ["bucket" as const] : []),
...(canDrive ? ["drive" as const] : [])];
      const showMediumPicker = mediumOptions.length > 1;
      const needsMediumUri = medium !== "local";
      // A non-local medium needs a destination URI before anything can be stored/submitted.
      const mediumBlocked = needsMediumUri && mediumUri.trim() === "";

      // Store the obtained source (an uploaded local path, or a pasted at-rest URI) INTO
the chosen medium,
      // then process the durable copy. A failed store is loud (surfaced like every other
ingest error).
      const storeThenProcess = async (src: { uploadedPath?: string; sourceUri?: string },
displayName?: string) => {
        const cid = newCorrelationId();
        setStoreError(null);
        setStoring(true);
        try {
          const asset = await createAsset({ ...src, mediumUri: mediumUri.trim() }, cid);
          log("info", "asset.stored", { cid, video_id: asset.video_id, medium: asset.medium,
copied: asset.copied });
          submit(asset.stored_uri, locale, sendCompute, displayName ??
sourceLabel(asset.stored_uri) ?? undefined); // index the stored copy
        } catch (err) {
          setStoreError(classifyError(err));
          log("error", "asset.store_failed", { cid, error: String(err) });
        } finally {
          setStoring(false);
        }
      };

      // On upload completion: local medium ? process the upload directly (today's flow); a
non-local
      // medium ? store a durable copy into it first, then process. The arrow is recreated
each render
      // (useUpload reads onCompleteRef.current), so `medium`/`mediumUri`/`sendCompute` are
always fresh.
      const upload = useUpload((path, filename) => {
        if (medium === "local") submit(path, locale, sendCompute, filename);
        else void storeThenProcess({ uploadedPath: path }, filename);
      }, maxUploadBytes);
      const busy = ingestBusy || upload.busy || storing;
      useEffect(() => {
        onBusyChange?.(busy);
        return () => {
```

```
169           if (busy) onBusyChange?.(false);
170         };
171       }, [busy, onBusyChange]);
172       // Uploading a FILE into a non-local medium needs a destination first. A pasted
cloud/host link is
173       // already an at-rest SOURCE and should process directly regardless of the storage
picker state.
174       const fileInputsDisabled = busy || cloudBlocked || mediumBlocked;
175       const uriInputsDisabled = busy || cloudBlocked;
176
177       const onSubmitUri = (e: FormEvent) => {
178         e.preventDefault();
179         if (uriInputsDisabled) return;
180         // The URI form is always a SOURCE (a bucket URL / host path to index directly). The
medium picker
181         // only controls where a locally picked file is stored before indexing.
182         const displayName = sourceLabel(uri);
183         setSelectedFile(null);
184         setActiveSource(displayName);
185         submit(uri, locale, sendCompute, displayName ?? undefined);
186       };
187       const onFile = (e: ChangeEvent<HTMLInputElement>) => {
188         const file = e.target.files?.[0];
189         // Symmetric with onSubmitUri: never accept a pick while blocked (the disabled input
already
190         // prevents this). Picking only arms the upload; the user starts it explicitly.
191         if (file && !fileInputsDisabled) {
192           upload.reset();
193           resetIngest();
194           setStoreError(null);
195           setSelectedFile(file);
196           setActiveSource(file.name);
197         }
198         e.target.value = ""; // let re-picking the same file fire change again
199       };
200       const startSelectedFile = () => {
201         if (!selectedFile || fileInputsDisabled) return;
202         setActiveSource(selectedFile.name);
203         upload.start(selectedFile);
204       };
205       const clearSelectedFile = () => {
206         setSelectedFile(null);
207         setActiveSource(null);
208         setStoreError(null);
209         upload.reset();
210         resetIngest();
211       };
212       const cancelAll = () => {
213         upload.cancel();
214         cancelIngest();
215         setSelectedFile(null);
216         setActiveSource(null);
217       };
218       const resetAll = () => {
219         upload.reset();
220         resetIngest();
221         setStoreError(null);
222         setSelectedFile(null);
223         setActiveSource(null);
224       };
225
226       // Shared mapping with the build flow (lib/errorText): an `engine` failure carries the
engine's own
227       // message (a 400 scheme/path/extension detail, a worker error); every other code maps
to a friendly
```

```
228       // localized line. Surface whichever half failed (upload or engine) ? they're mutually
exclusive.
229       const errorState =
230         upload.phase.status === "error"
231           ? upload.phase.error
232           : storeError ?? (phase.status === "error" ? phase.error : null);
233       const errorMsg = errorState ? errorText(t, errorState) : null;
234       const visibleSource = upload.phase.status === "uploading" ? upload.phase.filename :
activeSource;
235       const processingName =
236         phase.status === "submitting" || phase.status === "polling" ? phase.displayName :
undefined;
237       // Keep the video's name after a FAILURE too, so the retry screen still names it
(#161) ? including
238       // after a reload, when selectedFile/activeSource are gone and only the restored job
carries the name.
239       const failedName = phase.status === "error" ? phase.displayName : undefined;
240       const currentVideoName = selectedFile?.name ?? visibleSource ?? processingName ??
failedName;
241       const chooseFileLabel =
242         medium === "bucket" ? t("ingest.chooseFileBucket") : medium === "drive" ?
t("ingest.chooseFileDrive") : t("ingest.chooseFile");
243       const uploadAria =
244         medium === "bucket" ? t("ingest.uploadAriaBucket") : medium === "drive" ?
t("ingest.uploadAriaDrive") : t("ingest.uploadAria");
245       const fileHint =
246         medium === "bucket" ? t("ingest.fileHintBucket") : medium === "drive" ?
t("ingest.fileHintDrive") : t("ingest.fileHint");
247       const uploadEta = upload.phase.status === "uploading" ? formatEta(upload.phase.etaSec)
: null;
248
249       return (
250         <section className="mt-6 rounded-2xl border border-black/10 bg-white p-5" data-
testid="ingest-panel">
251           <h2 className="text-lg font-bold">{t("ingest.title")}</h2>
252           <p className="mt-1 max-w-2xl text-sm text-ink/60">{t("ingest.intro")}</p>
253           {currentVideoName && (
254             <div className="mt-4 max-w-2xl border-l-4 border-green pl-4" data-
testid="ingest-video-title">
255               <p className="text-xs font-semibold uppercase tracking-wide text-
green">{t("ingest.videoTitleLabel")}</p>
256               <p className="mt-1 break-words text-xl font-bold leading-snug text-
ink">{currentVideoName}</p>
257             </div>
258           )}
259
260           {!busy && (
261             <>
262             {/* UPLOAD-MOMENT DISCLOSURE (khelsutra #154): on a cloud-capable server (the
hosted alpha) the
263                 footage leaves the device, so testers get an honest, plain-English
disclosure of where it's
264                 stored + a link to the full policy. Suppressed on a local appliance
(cloudAvailable=false),
265                 where nothing is uploaded. Copy: ALPHA_LAUNCH_REVIEW_2026-07-04.md §4.4. */}
266             {cloudAvailable && (
267             <div
268               className="mt-4 max-w-2xl rounded-xl border border-amber-300/70 bg-amber-50
p-3 text-sm"
269               data-testid="upload-disclosure"
270             >
271               <p className="font-semibold text-ink/80">{t("ingest.disclosure.title")}</p>
272               <p className="mt-1 text-ink/70">{t("ingest.disclosure.body")}</p>
273               <a
274                 href="https://khelsutra.guru/privacy"
```

```
                target="_blank"
                rel="noreferrer"
                className="mt-2 inline-flex min-h-11 items-center font-medium text-green
underline"
              >
                {t("ingest.disclosure.link")}
              </a>
            </div>
            )}

            {/* COMPUTE PICKER (item 3): shown only when this server can dispatch to a cloud
GPU. Default
                LOCAL; CLOUD reveals a cost note + a required acknowledgement before any
submit can run. */}
            {cloudAvailable && (
            <fieldset className="mt-4 rounded-xl border border-black/10 p-3" data-
testid="compute-picker">
              <legend className="px-1 text-xs font-semibold uppercase tracking-wide text-
ink/40">
                {t("ingest.compute.legend")}
              </legend>
              <div className="mt-1 flex flex-wrap gap-2">
                {(["local", "cloud"] as const).map((opt) => (
                  <button
                    key={opt}
                    type="button"
                    aria-pressed={compute === opt}
                    disabled={busy}
                    onClick={() => setCompute(opt)}
                    className={
                      "min-h-11 rounded-xl border px-4 py-2 text-sm font-semibold
disabled:opacity-40 " +
                      (compute === opt
                        ? "border-green bg-green/10 text-ink"
                        : "border-black/15 text-ink/70 hover:border-green")
                    }
                  >
                    {t(`ingest.compute.${opt}`)}
                  </button>
                ))}
              </div>
              {compute === "cloud" && (
                <div className="mt-3" data-testid="compute-cost">
                  <p className="text-xs text-ink/60">{t("ingest.compute.cloudCost")}</p>
                  <label className="mt-2 flex items-center gap-2 text-sm text-ink/80">
                    <input
                      type="checkbox"
                      checked={cloudAck}
                      disabled={busy}
                      onChange={(e) => setCloudAck(e.target.checked)}
                      className="h-4 w-4"
                    />
                    {t("ingest.compute.ack")}
                  </label>
                  {cloudBlocked && (
                    <p className="mt-1 text-xs text-amber-700" data-testid="compute-ack-
required">
                      {t("ingest.compute.ackRequired")}
                    </p>
                  )}
                </div>
              )}
            </fieldset>
            )}

```

```
333             {/* MEDIUM PICKER (P5, D14): how the user wants to add this video. A non-local
medium still
334                 stores a durable MD5-keyed copy before indexing, but the copy uses user-
facing source
335                 language because the operator is choosing the route into Studio, not
thinking in backend
336                 storage planes. */}
337             {showMediumPicker && (
338             <fieldset className="mt-4 rounded-xl border border-black/10 p-3" data-
testid="medium-picker">
339               <legend className="px-1 text-xs font-semibold uppercase tracking-wide text-
ink/40">
340                 {t("ingest.medium.legend")}
341               </legend>
342               <div className="mt-1 flex flex-wrap gap-2">
343                 {mediumOptions.map((opt) => (
344                   <button
345                     key={opt}
346                     type="button"
347                     aria-pressed={medium === opt}
348                     disabled={busy}
349                     onClick={() => {
350                       setMedium(opt);
351                       setStoreError(null);
352                     }}
353                     data-testid={`medium-${opt}`}
354                     className={
355                       "min-h-11 rounded-xl border px-4 py-2 text-sm font-semibold
disabled:opacity-40 " +
356                       (medium === opt
357                         ? "border-green bg-green/10 text-ink"
358                         : "border-black/15 text-ink/70 hover:border-green")
359                     }
360                   >
361                     {t(`storage.${opt}`)}
362                   </button>
363                 ))}
364               </div>
365               {needsMediumUri && (
366                 <div className="mt-3" data-testid="medium-dest">
367                   <input
368                     type="text"
369                     value={mediumUri}
370                     onChange={(e) => setMediumUri(e.target.value)}
371                     placeholder={medium === "drive" ? t("ingest.medium.drivePlaceholder") :
t("ingest.medium.bucketPlaceholder")}
372                     aria-label={t("ingest.medium.destAria")}
373                     disabled={busy}
374                     className="min-h-11 w-full max-w-md rounded-xl border border-black/15
bg-white px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
375                   />
376                   <p className="mt-2 text-xs text-ink/50">{t("ingest.medium.destHint")}</p>
377                   {mediumBlocked && (
378                     <p className="mt-1 text-xs text-amber-700" data-testid="medium-
required">
379                       {t("ingest.medium.destRequired")}
380                     </p>
381                   )}
382                 </div>
383               )}
384             </fieldset>
385             )}
386
387             {/* PRIMARY: upload a local file (T4). A native file input, visually a button
(?44px, A4). */}
```

```
            <div className="mt-4">
            <label
              className={
                // focus-within makes the visible button show a focus ring when the (sr-
only) input is
                // focused ? a keyboard user needs a visible focus indicator (WCAG 2.4.7).
                "inline-flex min-h-11 items-center rounded-xl bg-ink px-4 py-2 text-sm font-
semibold text-white hover:bg-ink2 focus-within:outline-none focus-within:ring-2 focus-
within:ring-green focus-within:ring-offset-2 " +
                (fileInputsDisabled ? "pointer-events-none opacity-40" : "cursor-pointer")
              }
            >
              {chooseFileLabel}
              <input
                type="file"
                accept=".mp4,.mov,video/mp4,video/quicktime"
                aria-label={uploadAria}
                disabled={fileInputsDisabled}
                onChange={onFile}
                className="sr-only"
              />
            </label>
            <p className="mt-2 text-xs text-ink/40">{fileHint}</p>
            {selectedFile && (
              <div className="mt-3 max-w-2xl" data-testid="selected-file">
                <p className="text-xs font-medium text-ink/50">
                  {t("ingest.selectedFileReady", { size: formatUploadSize(selectedFile.size)
})}
                </p>
                <div className="mt-2 flex flex-wrap gap-2">
                  <button
                    type="button"
                    onClick={startSelectedFile}
                    disabled={fileInputsDisabled}
                    className="inline-flex min-h-11 items-center rounded-xl bg-ink px-4 py-2
text-sm font-semibold text-white disabled:opacity-40"
                  >
                    {t("ingest.startSelectedUpload")}
                  </button>
                  <button
                    type="button"
                    onClick={clearSelectedFile}
                    className="inline-flex min-h-11 items-center rounded-xl border border-
black/15 px-4 py-2 text-sm hover:border-green"
                  >
                    {t("ingest.clearSelectedFile")}
                  </button>
                </div>
              </div>
            )}
            </div>

            {/* SECONDARY: point at an existing cloud bucket / host path and index it
directly. This stays
                visible in every medium mode so a gs:// source is never confused with an
upload destination. */}
            <div className="mt-4 border-t border-black/5 pt-4">
            <p className="text-xs font-semibold uppercase tracking-wide text-
ink/40">{t("ingest.orLink")}</p>
            <form onSubmit={onSubmitUri} className="mt-2 flex flex-wrap items-center gap-2">
              <input
                type="text"
                value={uri}
                onChange={(e) => setUri(e.target.value)}
                placeholder={t("ingest.uriPlaceholder")}
```

```
444                 aria-label={t("ingest.uriAriaLabel")}
445                 disabled={uriInputsDisabled}
446                 className="min-h-11 min-w-[18rem] flex-1 rounded-xl border border-black/15
bg-white px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
447               />
448               <button
449                 type="submit"
450                 disabled={uriInputsDisabled}
451                 className="min-h-11 rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-
white disabled:opacity-40"
452               >
453                 {phase.status === "submitting" ? t("ingest.submitting") :
t("ingest.submit")}
454               </button>
455             </form>
456             <p className="mt-2 text-xs text-ink/40">{t("ingest.hint")}</p>
457             </div>
458             </>
459           )}
460
461           {/* Live status (aria-live so screen readers hear progress) as icon+text chips ?
never icon-only
462               (A8). Upload shows a determinate % bar; the engine phase is indeterminate
(phase words). */}
463           <div
464             aria-live="polite"
465             className={(busy ? "mt-4 border-t border-black/10 pt-4 " : "mt-3 ") + "text-sm"}
466             data-testid="ingest-status"
467           >
468             {upload.phase.status === "uploading" && (
469               <div data-testid="upload-progress">
470                 {/* Static chip label (not the per-tick %) so the aria-live region announces
once; the live
471                     % rides the progressbar's aria-valuenow + a sighted-only label (A8/T4).
*/}
472                 <StatusChip tone="info" icon={<Spinner />}
label={t("ingest.uploadingStatus")} />
473                 <p className="mt-2 text-xs font-medium text-ink/70" data-testid="ingest-
source">
474                   {t("ingest.sourceFile", { filename: upload.phase.filename })}
475                 </p>
476                 <p className="mt-1 text-xs text-ink/50" data-testid="upload-meta">
477                   {t("ingest.uploadMeta", {
478                     sent: formatUploadSize(upload.phase.sentBytes),
479                     total: formatUploadSize(upload.phase.sizeBytes),
480                     eta: uploadEta ? t("ingest.uploadEta", { time: uploadEta }) :
t("ingest.uploadEtaEstimating"),
481                   })}
482                 </p>
483                 <div className="mt-2 flex max-w-sm items-center gap-2">
484                   <div
485                     className="h-1.5 flex-1 overflow-hidden rounded-full bg-black/10"
486                     role="progressbar"
487                     aria-label={t("ingest.uploadProgress")}
488                     aria-valuenow={upload.phase.pct}
489                     aria-valuemin={0}
490                     aria-valuemax={100}
491                   >
492                     <div className="h-full bg-green transition-all" style={{ width:
`${upload.phase.pct}%` }} />
493                   </div>
494                   <span aria-hidden="true" className="text-xs tabular-nums text-ink/50">
495                     {upload.phase.pct}%
496                   </span>
497                 </div>
```

```
                <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
                <button
                  type="button"
                  onClick={cancelAll}
                  className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
                >
                  {t("ingest.cancel")}
                </button>
              </div>
            )}
            {storing && (
              <div data-testid="ingest-storing">
                <StatusChip tone="info" icon={<Spinner />}
label={t("ingest.medium.storing")} />
                {visibleSource && (
                  <p className="mt-2 text-xs font-medium text-ink/70" data-testid="ingest-
source">
                    {t("ingest.sourceFile", { filename: visibleSource })}
                  </p>
                )}
              </div>
            )}
            {ingestBusy && (
              <div>
                <StatusChip
                  tone="info"
                  icon={<Spinner />}
                  label={
                    t("ingest.working") +
                    (processingName ? ` · ${processingName}` : "") +
                    (phase.status === "polling" && phase.progress
                      ? ` · ${t("ingest.progress", { progress: phase.progress })}`
                      : "")
                  }
                />
                {visibleSource && (
                  <p className="mt-2 text-xs font-medium text-ink/70" data-testid="ingest-
source">
                    {t("ingest.sourceFile", { filename: visibleSource })}
                  </p>
                )}
                {phase.status === "polling" && (
                  <div className="mt-2 space-y-1 text-xs text-ink/50" data-testid="ingest-
heartbeat">
                    {phase.resumed && <p>{t("ingest.resumeNotice")}</p>}
                    <p>
                      {t("ingest.elapsed", { elapsed: formatElapsed(phase.elapsedSec) })} ·
{t("ingest.heartbeat")}
                    </p>
                    <p>{t("ingest.reloadHint")}</p>
                  </div>
                )}
                {/* Reassurance for slow uplinks: the job runs on the server, so it's safe
to wait. */}
                <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
                <button
                  type="button"
                  onClick={cancelAll}
                  className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
                >
                  {t("ingest.cancel")}
                </button>
              </div>
```

```
555             )}
556             {errorMsg && (
557               <div className="rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-
red-800" data-testid="ingest-error">
558                 <StatusChip tone="error" icon="?" label={t("ingest.statusFailed")} />
559                 {/* Name the failed video right on the retry screen so it's never
lost/untracked (#161). */}
560                 {currentVideoName && (
561                   <p className="mt-2 text-sm font-semibold text-red-900" data-
testid="ingest-source">
562                     {t("ingest.sourceFile", { filename: currentVideoName })}
563                   </p>
564                 )}
565                 <p className="mt-2">{errorMsg}</p>
566                 {/* Oversized upload: a coach may never have trimmed a clip ? offer
concrete, non-technical
567                     "how do I trim it?" steps inline (the appliance is air-gapped, so no
links out). Native
568                     <details> is keyboard-accessible and needs no JS. */}
569                 {errorState?.code === "fileTooLarge" && (
570                   <details className="mt-3 text-sm" data-testid="trim-help">
571                     <summary className="cursor-pointer font-medium text-red-900">
572                       {t("errors.fileTooLargeHelp.summary")}
573                     </summary>
574                     <ul className="mt-2 list-disc space-y-1 pl-5 text-red-900/80">
575                       <li>
576                         <Trans i18nKey="errors.fileTooLargeHelp.phone" components={{ b:
<strong /> }} />
577                       </li>
578                       <li>
579                         <Trans i18nKey="errors.fileTooLargeHelp.computer" components={{ b:
<strong /> }} />
580                       </li>
581                       <li>
582                         <Trans i18nKey="errors.fileTooLargeHelp.advanced" components={{ b:
<strong /> }} />
583                       </li>
584                     </ul>
585                   </details>
586                 )}
587                 <button
588                   type="button"
589                   onClick={resetAll}
590                   className="mt-2 inline-flex min-h-11 items-center rounded-lg border
border-red-300 px-3 text-sm hover:bg-red-100"
591                 >
592                   {t("ingest.retry")}
593                 </button>
594               </div>
595             )}
596           </div>
597         </section>
598       );
599     }
600
```

30. assistant (2026-07-07T15:12:09.390Z)

Now let me read the existing IngestPanel.test.tsx to check helpers and whether the default-to-cloud change breaks any EXISTING test (e.g. a test that renders with `cloud_run` caps and expects `local`).

31. user (2026-07-07T15:12:10.639Z)

```
1       // @vitest-environment jsdom
```

```
2       //
3       // The ingest screen end to end: type a URI ? submit ? (mocked engine) ? onLoaded gets
the video_id;
4       // and the two failure surfaces a non-technical user actually hits ? the engine's own
400 detail,
5       // and a friendly "is the engine running?" when it's unreachable. Mounts the real i18n
catalog.
6       import { describe, it, expect, vi, beforeEach, afterEach } from "vitest";
7       import { render, screen, fireEvent, cleanup, act, waitFor } from "@testing-
library/react";
8       import i18n from "../i18n/index";
9       import { IngestPanel } from "./IngestPanel";
10      import type { JobStatus } from "../api/types";
11
12      type FetchFn = typeof fetch;
13
14      function httpOk(status: number, json: unknown) {
15        return { ok: status >= 200 && status < 300, status, json: async () => json, text:
async () => JSON.stringify(json) };
16      }
17
18      function fetchSeq(process: unknown, jobs: JobStatus[]): FetchFn {
19        let i = 0;
20        return vi.fn(async (url: string | URL, init?: RequestInit) => {
21          const u = String(url);
22          // The panel probes /api/capabilities on mount (compute-picker) ? answer it benignly
so the
23          // ingest-flow assertions don't see a stray "unexpected fetch". No cloud ? no
picker.
24          if (u === "/api/capabilities") return httpOk(200, {});
25          if (u === "/api/process" && (init?.method ?? "GET") === "POST") return process;
26          if (u.startsWith("/api/jobs/")) {
27            const body = jobs[Math.min(i, jobs.length - 1)];
28            i += 1;
29            return httpOk(200, body);
30          }
31          throw new Error(`unexpected fetch ${u}`);
32        }) as unknown as FetchFn;
33      }
34
35      /** Like fetchSeq but lets a test set the capabilities payload + capture the
/api/process body ? used
36       *  to drive and assert the compute-picker. */
37      function fetchAll(opts: {
38        caps?: unknown;
39        process?: unknown;
40        jobs?: JobStatus[];
41        onProcessBody?: (body: Record<string, unknown>) => void;
42      }): FetchFn {
43        let i = 0;
44        const jobs = opts.jobs ?? [];
45        return vi.fn(async (url: string | URL, init?: RequestInit) => {
46          const u = String(url);
47          if (u === "/api/capabilities") return httpOk(200, opts.caps ?? {});
48          if (u === "/api/process" && (init?.method ?? "GET") === "POST") {
49            opts.onProcessBody?.(JSON.parse(String(init?.body ?? "{}")) as Record<string,
unknown>);
50            return opts.process ?? httpOk(202, { job_id: "j", status: "queued", poll:
"/api/jobs/j" });
51          }
52          if (u.startsWith("/api/jobs/")) {
53            const body = jobs[Math.min(i, jobs.length - 1)];
54            i += 1;
55            return httpOk(200, body);
56          }
```

```
57          throw new Error(`unexpected fetch ${u}`);
58        }) as unknown as FetchFn;
59      }
60
61      beforeEach(async () => {
62        window.localStorage.clear();
63        window.history.replaceState(null, "", "/");
64        await i18n.changeLanguage("en"); // guarantees init completed + a known locale for the
assertions
65      });
66
67      afterEach(() => {
68        cleanup();
69        vi.restoreAllMocks();
70        vi.unstubAllGlobals();
71        vi.useRealTimers();
72      });
73
74      const typeUri = (value: string) =>
75        fireEvent.change(screen.getByLabelText(/video location/i), { target: { value } });
76      const clickProcess = () => fireEvent.click(screen.getByRole("button", { name: /process
video/i }));
77      const clickStartUpload = () => fireEvent.click(screen.getByRole("button", { name: /start
upload and find rallies/i }));
78
79      describe("IngestPanel", () => {
80        it("submits a URI, polls, and hands the video_id to onLoaded", async () => {
81          vi.useFakeTimers();
82          vi.stubGlobal(
83            "fetch",
84            fetchSeq(httpOk(202, { job_id: "j", status: "queued", poll: "/api/jobs/j" }), [
85              { job_id: "j", status: "running", progress: "segmenting" },
86              { job_id: "j", status: "done", video_id: "vidZ", result: { status: "success",
video_id: "vidZ" } },
87            ]),
88          );
89          const onLoaded = vi.fn();
90          render(<IngestPanel onLoaded={onLoaded} />);
91          typeUri("gs://bucket/match.mp4");
92          clickProcess();
93
94          await act(async () => {
95            await vi.advanceTimersByTimeAsync(0); // flush submit ? start polling
96          });
97          await act(async () => {
98            await vi.advanceTimersByTimeAsync(1000); // running tick
99          });
100         await act(async () => {
101           await vi.advanceTimersByTimeAsync(1500); // done tick
102         });
103         expect(onLoaded).toHaveBeenCalledWith("vidZ", "match.mp4");
104       });
105
106       it("shows the engine's own 400 detail (e.g. an unsupported scheme)", async () => {
107         vi.stubGlobal("fetch", fetchSeq(httpOk(400, { detail: "unsupported storage URI
scheme: ftp://" }), []));
108         render(<IngestPanel onLoaded={vi.fn()} />);
109         typeUri("ftp://nope");
110         clickProcess();
111         expect(await screen.findByText(/unsupported storage URI scheme/i)).toBeTruthy();
112         fireEvent.click(screen.getByRole("button", { name: /try again/i }));
113         expect(screen.queryByTestId("ingest-error")).toBeNull();
114       });
115
116       it("shows a failed pipeline verdict as the engine's reason, not as no rallies", async
```

```
() => {
117         vi.useFakeTimers();
118         vi.stubGlobal(
119           "fetch",
120           fetchSeq(httpOk(202, { job_id: "jFail", status: "queued", poll: "/api/jobs/jFail"
}), [
121             { job_id: "jFail", status: "done", video_id: "vidFail", result: { status:
"failed", message: "Video failed validation: blur: blurry" } },
122           ]),
123         );
124         render(<IngestPanel onLoaded={vi.fn()} />);
125         typeUri("gs://bucket/blurry.mp4");
126         clickProcess();
127         await act(async () => {
128           await vi.advanceTimersByTimeAsync(0);
129         });
130         await act(async () => {
131           await vi.advanceTimersByTimeAsync(1000);
132         });
133         expect(screen.getByTestId("ingest-error").textContent).toMatch(/Video failed
validation: blur: blurry/i);
134         expect(screen.getByTestId("ingest-error").textContent).not.toMatch(/found no
rallies/i);
135       });
136
137       it("shows a friendly, localized message when the engine is unreachable", async () => {
138         vi.stubGlobal("fetch", vi.fn(async () => {
139           throw new Error("offline");
140         }) as unknown as FetchFn);
141         render(<IngestPanel onLoaded={vi.fn()} />);
142         typeUri("gs://bucket/match.mp4");
143         clickProcess();
144         expect(await screen.findByText(/can't reach khelsutra/i)).toBeTruthy();
145       });
146
147       it("rejects an empty URI with a hint and never submits to the engine", () => {
148         // The mount-time capabilities probe may fire; assert specifically that /api/process
never does.
149         const f = fetchAll({ caps: {} });
150         vi.stubGlobal("fetch", f);
151         render(<IngestPanel onLoaded={vi.fn()} />);
152         clickProcess();
153         expect(screen.getByTestId("ingest-error")).toBeTruthy();
154         const submitted = (f as unknown as { mock: { calls: unknown[][] }
}).mock.calls.some(
155           ([u]) => String(u) === "/api/process",
156         );
157         expect(submitted).toBe(false);
158       });
159
160       // --- compute-picker (item 3)
---------------------------------------------------------------------
161
162       it("hides the compute picker when the server can't dispatch to the cloud", async () =>
{
163         vi.stubGlobal("fetch", fetchAll({ caps: { deployment: { cloud_available: false } }
}));
164         render(<IngestPanel onLoaded={vi.fn()} />);
165         await act(async () => {}); // let the capabilities probe resolve
166         expect(screen.queryByTestId("compute-picker")).toBeNull();
167       });
168
169       it("defaults to local and sends compute=local when the cloud is available", async ()
=> {
170         const bodies: Record<string, unknown>[] = [];
```

```
171         vi.stubGlobal(
172           "fetch",
173           fetchAll({
174             caps: { deployment: { cloud_available: true } },
175             onProcessBody: (b) => bodies.push(b),
176             jobs: [{ job_id: "j", status: "done", video_id: "vidL", result: { status:
"success", video_id: "vidL" } }],
177           }),
178         );
179         render(<IngestPanel onLoaded={vi.fn()} />);
180         await screen.findByTestId("compute-picker");
181         typeUri("gs://bucket/match.mp4");
182         clickProcess();
183         await act(async () => {});
184         expect(bodies).toHaveLength(1);
185         expect(bodies[0].compute).toBe("local");
186       });
187
188       it("defaults to CLOUD (never sends compute=local) on a hosted cloud_run control-
plane", async () => {
189         // A hosted control-plane bakes no detector weights, so LOCAL can't run there ? the
picker must
190         // default to CLOUD. The cost gate still applies (accept it, then the sole submit is
compute=cloud).
191         const bodies: Record<string, unknown>[] = [];
192         vi.stubGlobal(
193           "fetch",
194           fetchAll({
195             caps: { deployment: { cloud_available: true, compute_target: "cloud_run" } },
196             onProcessBody: (b) => bodies.push(b),
197             jobs: [{ job_id: "j", status: "done", video_id: "vidH", result: { status:
"success", video_id: "vidH" } }],
198           }),
199         );
200         render(<IngestPanel onLoaded={vi.fn()} />);
201         await screen.findByTestId("compute-picker");
202         // Defaulted to cloud ? the cost note + ack are shown up front (LOCAL would hide
them).
203         expect(screen.getByTestId("compute-cost")).toBeTruthy();
204         typeUri("gs://bucket/match.mp4");
205         fireEvent.click(screen.getByRole("checkbox")); // accept the cost gate
206         clickProcess();
207         await act(async () => {});
208         expect(bodies).toHaveLength(1);
209         expect(bodies[0].compute).toBe("cloud"); // never "local" by default on the hosted
build
210       });
211
212       it("shows a pasted host-path basename while processing and clears it on cancel", async
() => {
213         vi.useFakeTimers();
214         vi.stubGlobal(
215           "fetch",
216           fetchAll({
217             caps: {},
218             jobs: [{ job_id: "j", status: "running", progress: "hashing" }],
219           }),
220         );
221         render(<IngestPanel onLoaded={vi.fn()} />);
222
223         typeUri("C:\\videos\\Sunday Match.mov");
224         clickProcess();
225         await act(async () => {
226           await vi.advanceTimersByTimeAsync(0);
227         });
```

```
228
229         expect(screen.getByTestId("ingest-source").textContent).toContain("Sunday
Match.mov");
230         fireEvent.click(screen.getByRole("button", { name: /cancel/i }));
231         expect(screen.queryByTestId("ingest-source")).toBeNull();
232       });
233
234       it("shows a decoded URL basename while processing", async () => {
235         vi.useFakeTimers();
236         vi.stubGlobal(
237           "fetch",
238           fetchAll({
239             caps: {},
240             jobs: [{ job_id: "j", status: "running", progress: "hashing" }],
241           }),
242         );
243         render(<IngestPanel onLoaded={vi.fn()} />);
244
245         typeUri("https://cdn.example/video/Sunday%20Final.mp4?download=1");
246         clickProcess();
247         await act(async () => {
248           await vi.advanceTimersByTimeAsync(0);
249         });
250
251         expect(screen.getByTestId("ingest-source").textContent).toContain("Sunday
Final.mp4");
252       });
253
254       // --- medium picker (P5, store-into-medium)
-------------------------------------------------
255
256       const capsBucket = {
257         storage: {
258           default_medium: "local",
259           backends: [
260             { id: "local", kind: "local", label: "This computer", usable: true, free_gb: 100
},
261             { id: "gcs", kind: "gcs", label: "GCS", usable: true, free_gb: null },
262             { id: "s3", kind: "s3", label: "S3", usable: false, free_gb: null },
263             { id: "gdrive", kind: "gdrive", label: "Drive", usable: false, free_gb: null },
264           ],
265         },
266       };
267
268       // A fetch mock that answers the full store?process choreography (upload ? assets ?
process ? job).
269       function fetchStore(opts: {
270         caps?: unknown;
271         jobs?: JobStatus[];
272         onAssetBody?: (b: Record<string, unknown>) => void;
273         onProcessBody?: (b: Record<string, unknown>) => void;
274         holdPartIndex?: number;
275         holdPart?: Promise<void>;
276         onPartRequest?: (part: number) => void;
277         maxPartMb?: number;
278       }): FetchFn {
279         let ji = 0;
280         const jobs = opts.jobs ?? [];
281         return vi.fn(async (url: string | URL, init?: RequestInit) => {
282           const u = String(url);
283           const m = init?.method ?? "GET";
284           if (u === "/api/capabilities") return httpOk(200, opts.caps ?? {});
285           if (u === "/api/upload/init") return httpOk(200, { upload_id: "up1", max_part_mb:
opts.maxPartMb ?? 95, next_part: "/api/upload/up1/part/0" });
286           const partMatch = u.match(/^\/api\/upload\/up1\/part\/(\d+)$/);
```

```
287           if (partMatch) {
288             const part = Number(partMatch[1]);
289             opts.onPartRequest?.(part);
290             if (opts.holdPartIndex === part) await opts.holdPart;
291             return httpOk(200, { upload_id: "up1", received_parts: part + 1, received_bytes:
(init?.body as Blob).size });
292           }
293           if (/^\/api\/upload\/up1\/complete$/.test(u)) return httpOk(200, { path:
"/srv/up/match.mp4", video_id: "mock", size_bytes: 3 });
294           if (u === "/api/assets" && m === "POST") {
295             opts.onAssetBody?.(JSON.parse(String(init?.body ?? "{}")) as Record<string,
unknown>);
296             return httpOk(201, { video_id: "vidS", medium: "gcs", stored_uri:
"gcs://b/videos/vidS.mp4", copied: true, status: "stored" });
297           }
298           if (u === "/api/process" && m === "POST") {
299             opts.onProcessBody?.(JSON.parse(String(init?.body ?? "{}")) as Record<string,
unknown>);
300             return httpOk(202, { job_id: "j", status: "queued", poll: "/api/jobs/j" });
301           }
302           if (u.startsWith("/api/jobs/")) {
303             const b = jobs[Math.min(ji, jobs.length - 1)];
304             ji += 1;
305             return httpOk(200, b);
306           }
307           throw new Error(`unexpected fetch ${m} ${u}`);
308         }) as unknown as FetchFn;
309       }
310
311       it("hides the medium picker when only local is usable", async () => {
312         vi.stubGlobal("fetch", fetchAll({ caps: { storage: { backends: [{ id: "local", kind:
"local", label: "x", usable: true }] } } }));
313         render(<IngestPanel onLoaded={vi.fn()} />);
314         await act(async () => {});
315         expect(screen.queryByTestId("medium-picker")).toBeNull();
316       });
317
318       it("shows the picker and gates the upload until a destination is given", async () => {
319         vi.stubGlobal("fetch", fetchStore({ caps: capsBucket }));
320         render(<IngestPanel onLoaded={vi.fn()} />);
321         await screen.findByTestId("medium-picker");
322         fireEvent.click(screen.getByTestId("medium-bucket"));
323         // Destination empty ? blocked: the required hint shows and the file input is
disabled.
324         expect(screen.getByTestId("medium-required")).toBeTruthy();
325         const fileInput = document.querySelector('input[type="file"]') as HTMLInputElement;
326         expect(fileInput.disabled).toBe(true);
327         // Provide a destination ? the gate lifts.
328         fireEvent.change(screen.getByLabelText(/storage destination/i), { target: { value:
"gcs://b/videos" } });
329         expect(screen.queryByTestId("medium-required")).toBeNull();
330         expect(fileInput.disabled).toBe(false);
331         expect(screen.getByText(/choose video for bucket/i)).toBeTruthy();
332         expect(screen.getByLabelText(/selected cloud bucket/i)).toBe(fileInput);
333         expect(screen.getByText("MP4 or MOV.")).toBeTruthy();
334         // The source-link form stays visible: a gs:// source is not the same thing as the
upload destination.
335         expect(screen.getByText(/or use a cloud link/i)).toBeTruthy();
336       });
337
338       it("lets a pasted gs:// source submit even when bucket storage is selected without a
destination", async () => {
339         const assetBodies: Record<string, unknown>[] = [];
340         const processBodies: Record<string, unknown>[] = [];
341         vi.stubGlobal(
```

```
          "fetch",
          fetchStore({
            caps: capsBucket,
            onAssetBody: (b) => assetBodies.push(b),
            onProcessBody: (b) => processBodies.push(b),
            jobs: [{ job_id: "j", status: "done", video_id: "vidLink", result: { status:
"success", video_id: "vidLink" } }],
          }),
        );
        const onLoaded = vi.fn();
        render(<IngestPanel onLoaded={onLoaded} />);
        await screen.findByTestId("medium-picker");
        fireEvent.click(screen.getByTestId("medium-bucket"));
        expect(screen.getByTestId("medium-required")).toBeTruthy();

        typeUri("gs://source-bucket/matches/final.mp4");
        clickProcess();

        await waitFor(() => expect(processBodies).toHaveLength(1));
        expect(processBodies[0].video_path).toBe("gs://source-bucket/matches/final.mp4");
        expect(assetBodies).toHaveLength(0);
      });

      it("replaces setup controls with upload status, size and ETA once a bucket upload
starts", async () => {
        let releasePart: () => void = () => {};
        const heldPart = new Promise<void>((resolve) => {
          releasePart = resolve;
        });
        let secondPartStarted: () => void = () => {};
        const secondPart = new Promise<void>((resolve) => {
          secondPartStarted = resolve;
        });
        vi.stubGlobal(
          "fetch",
          fetchStore({
            caps: { deployment: { cloud_available: true }, ...capsBucket },
            maxPartMb: 1,
            holdPartIndex: 1,
            holdPart: heldPart,
            onPartRequest: (part) => {
              if (part === 1) secondPartStarted();
            },
            jobs: [{ job_id: "j", status: "done", video_id: "vidS", result: { status:
"success", video_id: "vidS" } }],
          }),
        );
        render(<IngestPanel onLoaded={vi.fn()} />);
        await screen.findByTestId("compute-picker");
        fireEvent.click(screen.getByTestId("medium-bucket"));
        fireEvent.change(screen.getByLabelText(/storage destination/i), { target: { value:
"gcs://b/videos" } });
        const file = new File([new Uint8Array(2 * 1024 * 1024)], "match.mp4", { type:
"video/mp4" });

        await act(async () => {
          fireEvent.change(document.querySelector('input[type="file"]') as HTMLInputElement,
{ target: { files: [file] } });
        });

        expect(screen.getByTestId("ingest-video-title").textContent).toContain("match.mp4");
        expect(screen.getByTestId("selected-file").textContent).toContain("2 MB selected");
        expect(screen.queryByTestId("upload-progress")).toBeNull();
        expect(screen.getByTestId("compute-picker")).toBeTruthy();
        expect(screen.getByTestId("medium-picker")).toBeTruthy();
```

```

        await act(async () => {
          clickStartUpload();
          await secondPart;
        });

        expect(screen.getByTestId("upload-progress")).toBeTruthy();
        expect(screen.queryByTestId("compute-picker")).toBeNull();
        expect(screen.queryByTestId("medium-picker")).toBeNull();
        expect(screen.queryByText(/or use a cloud link/i)).toBeNull();
        expect(screen.getByTestId("upload-meta").textContent).toMatch(/1 MB of 2 MB uploaded
· 0:0\d left/);

        await act(async () => {
          releasePart();
        });
      });

      it("offers Drive as a storage medium when the engine reports a usable mount", async ()
=> {
        vi.stubGlobal(
          "fetch",
          fetchStore({
            caps: {
              storage: {
                backends: [
                  { id: "local", kind: "local", label: "This computer", usable: true },
                  { id: "gdrive", kind: "gdrive", label: "Drive", usable: true },
                ],
              },
            },
          }),
        );
        render(<IngestPanel onLoaded={vi.fn()} />);
        await screen.findByTestId("medium-picker");
        fireEvent.click(screen.getByTestId("medium-drive"));
        expect(screen.getByPlaceholderText("gdrive://Matches")).toBeTruthy();
      });

      it("stores an uploaded video into the chosen medium, then processes the stored copy",
async () => {
        const assetBodies: Record<string, unknown>[] = [];
        const processBodies: Record<string, unknown>[] = [];
        vi.stubGlobal(
          "fetch",
          fetchStore({
            caps: capsBucket,
            onAssetBody: (b) => assetBodies.push(b),
            onProcessBody: (b) => processBodies.push(b),
            jobs: [{ job_id: "j", status: "done", video_id: "vidS", result: { status:
"success", video_id: "vidS" } }],
          }),
        );
        render(<IngestPanel onLoaded={vi.fn()} />);
        await screen.findByTestId("medium-picker");
        fireEvent.click(screen.getByTestId("medium-bucket"));
        fireEvent.change(screen.getByLabelText(/storage destination/i), { target: { value:
"gcs://b/videos" } });
        const file = new File([new Uint8Array([1, 2, 3])], "match.mp4", { type: "video/mp4"
});
        const fileInput = document.querySelector('input[type="file"]') as HTMLInputElement;
        await act(async () => {
          fireEvent.change(fileInput, { target: { files: [file] } });
        });
        expect(screen.getByTestId("ingest-video-title").textContent).toContain("match.mp4");
```

```
460         expect(assetBodies).toHaveLength(0);
461         clickStartUpload();
462         // The upload ? createAsset ? process chain runs: the asset is stored MD5-keyed,
then the STORED
463         // uri (not the raw upload path) is what's indexed.
464         await waitFor(() => expect(assetBodies).toHaveLength(1));
465         expect(assetBodies[0]).toEqual({ uploaded_path: "/srv/up/match.mp4", medium_uri:
"gcs://b/videos" });
466         await waitFor(() => expect(processBodies).toHaveLength(1));
467         expect(processBodies[0].video_path).toBe("gcs://b/videos/vidS.mp4");
468       });
469
470       it("surfaces a store failure loudly (e.g. the engine's 507 no-space)", async () => {
471         const fetchFail = fetchStore({ caps: capsBucket });
472         // Override /api/assets to a 507.
473         const wrapped = vi.fn(async (url: string | URL, init?: RequestInit) => {
474           if (String(url) === "/api/assets") return httpOk(507, { detail: "Not enough free
disk space for store" });
475           return (fetchFail as unknown as (u: string | URL, i?: RequestInit) =>
Promise<unknown>)(url, init);
476         }) as unknown as FetchFn;
477         vi.stubGlobal("fetch", wrapped);
478         render(<IngestPanel onLoaded={vi.fn()} />);
479         await screen.findByTestId("medium-picker");
480         fireEvent.click(screen.getByTestId("medium-bucket"));
481         fireEvent.change(screen.getByLabelText(/storage destination/i), { target: { value:
"gcs://b/videos" } });
482         const file = new File([new Uint8Array([1, 2, 3])], "match.mp4", { type: "video/mp4"
});
483         await act(async () => {
484           fireEvent.change(document.querySelector('input[type="file"]') as HTMLInputElement,
{ target: { files: [file] } });
485         });
486         clickStartUpload();
487         expect(await screen.findByTestId("ingest-error")).toBeTruthy();
488       });
489
490       it("gates cloud on a cost ack, then sends compute=cloud", async () => {
491         const bodies: Record<string, unknown>[] = [];
492         vi.stubGlobal(
493           "fetch",
494           fetchAll({
495             caps: { deployment: { cloud_available: true } },
496             onProcessBody: (b) => bodies.push(b),
497             jobs: [{ job_id: "j", status: "done", video_id: "vidC", result: { status:
"success", video_id: "vidC" } }],
498           }),
499         );
500         render(<IngestPanel onLoaded={vi.fn()} />);
501         await screen.findByTestId("compute-picker");
502         // Type the URI while LOCAL (the input is enabled), then switch to CLOUD.
503         typeUri("gs://bucket/match.mp4");
504         fireEvent.click(screen.getByRole("button", { name: /in the cloud/i }));
505         // Cost note + ack appear; submit is gated until the cost is accepted.
506         expect(screen.getByTestId("compute-cost")).toBeTruthy();
507         expect(screen.getByTestId("compute-ack-required")).toBeTruthy();
508         const submitBtn = screen.getByRole("button", { name: /process video/i }) as
HTMLButtonElement;
509         expect(submitBtn.disabled).toBe(true);
510         clickProcess(); // a disabled button does nothing ? still no submit
511         expect(bodies).toHaveLength(0);
512         // Accept the cost ? the gate lifts ? run on the cloud.
513         fireEvent.click(screen.getByRole("checkbox"));
514         expect(submitBtn.disabled).toBe(false);
515         clickProcess();
```

```
516         await act(async () => {});
517         expect(bodies).toHaveLength(1);
518         expect(bodies[0].compute).toBe("cloud");
519       });
520
521       it("shows the upload-moment privacy disclosure + a link to the full policy on a cloud
server (#154)", async () => {
522         vi.stubGlobal("fetch", fetchAll({ caps: { deployment: { cloud_available: true } }
}));
523         render(<IngestPanel onLoaded={vi.fn()} />);
524         const box = await screen.findByTestId("upload-disclosure");
525         // Honest about WHERE footage goes (the blocker this closes), and no "never leaves
your device" lie.
526         expect(box.textContent).toMatch(/Google Cloud bucket in Singapore/);
527         expect(box.textContent).not.toMatch(/never leave/i);
528         const link = screen.getByRole("link", { name: /privacy policy/i }) as
HTMLAnchorElement;
529         expect(link.getAttribute("href")).toBe("https://khelsutra.guru/privacy");
530         expect(link.className).toMatch(/\b(?:min-h-11|h-11)\b/); // real tap target (A4 /
WCAG 2.5.5)
531       });
532
533       it("suppresses the disclosure on a local appliance where nothing is uploaded (#154)",
async () => {
534         vi.stubGlobal("fetch", fetchAll({ caps: {} })); // no deployment.cloud_available ?
local-only
535         render(<IngestPanel onLoaded={vi.fn()} />);
536         await screen.findByTestId("ingest-panel");
537         expect(screen.queryByTestId("upload-disclosure")).toBeNull();
538       });
539
540       // --- #161: the failed video is always named on the retry screen
------------------------------
541       // Owner ask (issue #161 + comment): the rendered page must DETERMINISTICALLY show
which video the
542       // session is working on ? especially on the failure/retry screen, where the name was
being lost.
543
544       it("names the video on the retry screen when a single-shot upload fails (#161)", async
() => {
545         // Local medium, engine unreachable at upload/init ? the upload fails and the retry
card appears.
546         vi.stubGlobal(
547           "fetch",
548           vi.fn(async (url: string | URL) => {
549             const u = String(url);
550             if (u === "/api/capabilities") return httpOk(200, {});
551             if (u === "/api/upload/init") throw new Error("offline");
552             throw new Error(`unexpected fetch ${u}`);
553           }) as unknown as FetchFn,
554         );
555         render(<IngestPanel onLoaded={vi.fn()} />);
556         await screen.findByTestId("ingest-panel");
557         const file = new File([new Uint8Array([1, 2, 3])], "Quarterfinal.mp4", { type:
"video/mp4" });
558         await act(async () => {
559           fireEvent.change(document.querySelector('input[type="file"]') as HTMLInputElement,
{ target: { files: [file] } });
560         });
561         await act(async () => {
562           clickStartUpload();
563         });
564         // The retry card itself names the failed video (never a bare placeholder)...
565         const errorCard = await screen.findByTestId("ingest-error");
566         expect(errorCard.textContent).toContain("Quarterfinal.mp4");
```

```
567         // ...and the persistent title still shows it too.
568         expect(screen.getByTestId("ingest-video-
title").textContent).toContain("Quarterfinal.mp4");
569       });
570
571       it("names the video on the retry screen when a URI submission fails (#161)", async ()
=> {
572         vi.stubGlobal("fetch", fetchSeq(httpOk(400, { detail: "unsupported storage URI
scheme: ftp://" }), []));
573         render(<IngestPanel onLoaded={vi.fn()} />);
574         typeUri("gs://bucket/SemiFinal.mp4");
575         clickProcess();
576         const errorCard = await screen.findByTestId("ingest-error");
577         expect(errorCard.textContent).toContain("SemiFinal.mp4");
578       });
579
580       it("still names the video on the retry screen after a reload, once the resumed job
fails (#161)", async () => {
581         // The real gap: the page was reloaded mid-processing (component state gone), then
the resumed job
582         // fails. The name must survive via the restored job ? error phase ? retry card,
with NO prior
583         // in-page interaction. This fails on the old code (error phase dropped the display
name).
584         window.localStorage.setItem(
585           "khelsutra.activeIngestJob",
586           JSON.stringify({ jobId: "jR", uri: "gs://b/reload.mp4", cid: "c1", startedAt:
1000, displayName: "reload.mp4" }),
587         );
588         window.history.replaceState(null, "", "/?job=jR");
589         vi.stubGlobal(
590           "fetch",
591           vi.fn(async (url: string | URL) => {
592             const u = String(url);
593             if (u === "/api/capabilities") return httpOk(200, {});
594             if (u.startsWith("/api/jobs/")) return httpOk(200, { job_id: "jR", status:
"failed", error: "worker crashed" });
595             throw new Error(`unexpected fetch ${u}`);
596           }) as unknown as FetchFn,
597         );
598         render(<IngestPanel onLoaded={vi.fn()} />);
599         const errorCard = await screen.findByTestId("ingest-error");
600         expect(errorCard.textContent).toContain("reload.mp4");
601       });
602     });
603
```

32. assistant (2026-07-07T15:12:30.749Z)

Now let me verify the SPA type surface ? does `c.deployment?.compute_target` exist on the capabilities type? If not, `tsc` would fail the build. Let me check types.ts and the backend capabilities handler.

33. user (2026-07-07T15:12:34.393Z)

```
=== SPA types.ts: deployment/compute_target/cloud_available ===
92:export interface EngineCapabilities {
107:  /** `cloud_available` = can THIS server satisfy a per-request compute="cloud" dispatch (a
Cloud Run
109:  deployment?: { profile?: string; compute_target?: string; cloud_available?: boolean };
113:  storage?: StorageCapabilities;
128:export interface StorageCapabilities {

=== backend /api/capabilities emits compute_target? ===
235:    _cloud_available,
```

```
267:    # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
268:    # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
401:            "compute_target": getattr(deployment, "compute_target", "") or "",
404:            "cloud_available": _cloud_available(cfg),
```

34. assistant (2026-07-07T15:12:48.668Z)

Types check out: `compute_target?: string` is declared (types.ts:109), and the backend emits it (main.py:401). No TS regression. Now let me verify the new backend test's provenance assertions actually hold ? how `_execute_processing` consumes `_maybe_dispatch_remote`'s failed tuple.

35. user (2026-07-07T15:12:53.013Z)

```
=== call site of _maybe_dispatch_remote in _execute_processing ===
4:Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` / ``_ingest_remote_segments``),
161:# guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic deep
423:def _maybe_dispatch_remote(
450:        # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker rc /
ingest
452:        # intended backend, `requested` the picker choice, `dispatched` whether the remote
job ran.
464:                    "dispatched": ran,
681:        "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s video_id=%s",
770:    # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
847:        remote = _maybe_dispatch_remote(
912:            "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",

--- lines 840-915 ---
        # if the run yields segments; otherwise drop the new partial rows and keep the old.
        play_wm, cand_wm = db.segment_watermarks(video_id)
        # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run, run the
heavy
        # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
        # DEFAULT-OFF: returns None for every non-cloud target ? the in-process segmenter below
runs
        # byte-identically. (player_pool/max_frames aren't threaded to the worker `infer` CLI
yet, so
        # they're dropped on the cloud path in this slice ? a documented follow-up.)
        remote = _maybe_dispatch_remote(
            config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
        )
        if remote is not None:
            response, segmentation_ran = (
                remote  # `ran` = did the cloud job actually execute (report)
            )
            return response  # the cloud branch already stamped response["compute"]
(path=cloud_run)
        # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
        compute_actual = "in_process"
        segmenter = segmenter_cls(db, config)
        try:
            # Serialize the CPU/GPU-heavy in-process detection (see _LOCAL_COMPUTE_LANE). A
remote
            # dispatch never reaches here (it returned above), so remote polls still overlap
freely.
            with _LOCAL_COMPUTE_LANE:
                result = segmenter.process_video(
                    video_id, local_video, req.player_pool, req.max_frames
                )
        except Exception:
```

```
            # A raising segmenter must not leave half-written rows: restore the pre-run
            # state (same destroy-after-confirm contract as the Failure branch), then let
            # the job worker record the failure.
            db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
            raise
        segmentation_ran = True

        if isinstance(result, Failure):
            logger.error(f"[API] Segmenter failed: {result.message}")
            db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
            response = {
                "video_id": video_id,
                "status": "failed",
                "message": f"Segmenter '{seg_name}' failed: {result.message}",
                "results": [r.model_dump() for r in val_results],
                "play_segments": [],
            }
            return response

        segments = result.value if hasattr(result, "value") else result
        notify("finalizing")
        _finalize_reingest(db, video_id, segments, play_wm, cand_wm)

        response = {
            "video_id": video_id,
            "status": "success",
            "message": f"Successfully indexed {len(segments)} play segments using '{seg_name}'
segmenter.",  # type: ignore[arg-type]  # hasattr-unwrap above; Success arm has .value
            "results": [r.model_dump() for r in val_results],
            "play_segments": segments,
        }
        return response
    finally:
        # Compute-path PROVENANCE stamp (the validation method): the cloud branch already set
        # response["compute"] (path=cloud_run + the Cloud Run execution); for every other path
stamp
        # the actual path here so the job result PROVES which backend ran (in_process / not_run)
and
        # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is then
visible.
        if isinstance(response, dict):
            response.setdefault(
                "compute",
                {
                    "path": compute_actual or "not_run",
                    "requested": getattr(req, "compute", None),
                },
            )
        prov = response.get("compute", {}) if isinstance(response, dict) else {}
        logger.info(
            "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
            video_id,
            getattr(req, "compute", None),
            prov.get("path"),
```

36. assistant (2026-07-07T15:13:31.656Z)

The dispatch plumbing confirms the new tests' provenance assertions hold (`setdefault` preserves `_failed`'s compute dict). Now the crux for coverage/correctness: is the `_WEIGHTS_BACKED_SEGMENTERS` frozenset complete? The task mentions a `trajectory_hybrid` segmenter that's NOT in the set. Let me check the registered segmenter names and whether `trajectory_hybrid` is weights-backed.

37. user (2026-07-07T15:13:36.153Z)

```
=== segmenter files ===
ls: cannot access 'C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters"; echo; echo ===': No such file or directory
ls: cannot access 'registered': No such file or directory
ls: cannot access '@segmenter_registry': No such file or directory
ls: cannot access 'names': No such file or directory
ls: cannot access '===; grep -rnE register': No such file or directory
/usr/bin/bash: line 1: @segmenter_registry: command not found
/usr/bin/bash: line 1: _registry.register: command not found
/usr/bin/bash: line 1: names*=: command not found
/usr/bin/bash: line 1: SEGMENTER_NAME: command not found
/usr/bin/bash: line 1: register( C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backendpipelinesegmenters": No such file or directory
```

38. assistant (2026-07-07T15:13:40.556Z)

Windows backslash path issue. Let me use the dedicated tools.

39. user (2026-07-07T15:13:41.617Z)

No files found

40. user (2026-07-07T15:13:51.585Z)

```
=== backend/pipeline dir ===
audio
detectors
personalization
segmenters
stitcher
validators
=== segmenters ===
__init__.py
__pycache__
_keyhint.py
base.py
fusion_hybrid.py
gemini.py
motion.py
tracknet_hybrid.py
trajectory_hybrid.py
wasb_hybrid.py
yolo_hybrid.py
=== detectors ===
__init__.py
__pycache__
audio_features.py
base.py
court_gate.py
device.py
fusion_features.py
native_wasb_runner.py
person_detector.py
rally_gate.py
stub_runner.py
tracknet_runner.py
wasb_infer.py
wasb_runner.py
```

41. assistant (2026-07-07T15:14:03.211Z)

There are two more hybrids not in the frozenset: `trajectory_hybrid.py` and `yolo_hybrid.py`.
Let me check their registered names and whether they're weights-backed and user-selectable.

42. user (2026-07-07T15:14:05.239Z)

```
=== registration lines across segmenters ===
backend/pipeline/segmenters/base.py:66:    def register(self, segmenter_cls:
Type[BasePlaySegmenter]):
backend/pipeline/segmenters/fusion_hybrid.py:243:segmenter_registry.register(FusionSegmenter)
backend/pipeline/segmenters/gemini.py:707:segmenter_registry.register(GeminiPlaySegmenter)
backend/pipeline/segmenters/motion.py:243:segmenter_registry.register(MotionPlaySegmenter)
backend/pipeline/segmenters/tracknet_hybrid.py:48:segmenter_registry.register(TrackNetHybridSegm
enter)
backend/pipeline/segmenters/wasb_hybrid.py:68:segmenter_registry.register(WasbHybridSegmenter)
backend/pipeline/segmenters/yolo_hybrid.py:596:segmenter_registry.register(HybridYoloSegmenter)
```

43. assistant (2026-07-07T15:14:26.054Z)

`trajectory_hybrid.py` is NOT registered (likely a shared base); `yolo_hybrid.py` IS registered as `HybridYoloSegmenter`. Let me confirm the registered `name` identifiers match the frozenset, and check whether yolo is weights-backed and user-selectable.

44. user (2026-07-07T15:14:31.430Z)

```
=== 'name =' attr in each segmenter ===
backend/pipeline/segmenters/base.py:70:            name = temp_inst.name

=== what is trajectory_hybrid.py (class/registration)? ===
4:standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
10:A concrete subclass supplies only its identity and runner wiring:
18:This base is deliberately NOT registered; only concrete subclasses register.
50:class TrajectoryHybridSegmenter(BasePlaySegmenter):
190:        sport_name = self.config.sport
200:        provider_name = self.config.ai_provider
202:            model_name = "local-noai"
210:            model_name = self.config.ai_model
246:        # it delegates to the subclass _build_runner (the real WSL/native runner).

=== yolo_hybrid: weights / model loading ===
497:        self.person.lazy_load_model()
```

45. assistant (2026-07-07T15:14:41.992Z)

`trajectory_hybrid` is the deliberately-unregistered base ? the three subclasses in the frozenset are exactly right. `yolo_hybrid` uses a YOLO person detector, not WASB/TrackNet trajectory weights. Let me confirm the registered `name` identifiers (to verify the frozenset strings match) and whether yolo needs baked weights.

46. user (2026-07-07T15:14:45.824Z)

```
=== name identifiers (property/attr) ===
backend/pipeline/segmenters/wasb_hybrid.py:30:    def name(self) -> str:
backend/pipeline/segmenters/tracknet_hybrid.py:28:    def name(self) -> str:
backend/pipeline/segmenters/fusion_hybrid.py:57:    def name(self) -> str:
backend/pipeline/segmenters/yolo_hybrid.py:56:    def name(self) -> str:
backend/pipeline/segmenters/yolo_hybrid.py:75:        sport_name = self.config.sport
backend/pipeline/segmenters/yolo_hybrid.py:83:        provider_name = self.config.ai_provider
backend/pipeline/segmenters/yolo_hybrid.py:84:        model_name = self.config.ai_model
backend/pipeline/segmenters/yolo_hybrid.py:324:        provider_name: str,
backend/pipeline/segmenters/yolo_hybrid.py:434:        self, video_id: str, gemini_segments:
List[dict], model_name: str
backend/pipeline/segmenters/yolo_hybrid.py:507:        provider_name = self.config.ai_provider

=== how base gets name (around line 60-75) ===
        """
        pass
```

```
class SegmenterRegistry:
    def __init__(self):
        self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}

    def register(self, segmenter_cls: Type[BasePlaySegmenter]):
        """Registers a new segmenter class under its name property."""
        try:
            temp_inst = segmenter_cls(None, {})
            name = temp_inst.name
        except Exception as e:
            raise ValueError(
                f"Failed to resolve name property from segmenter class {segmenter_cls.__name__}:
{e}"
            ) from e
        self._segmenters[name] = segmenter_cls
```

47. user (2026-07-07T15:14:51.734Z)

```
=== wasb_hybrid name property ===
    def name(self) -> str:
        return "wasb_hybrid"

=== tracknet_hybrid name property ===
    def name(self) -> str:
        return "tracknet_hybrid"

=== fusion_hybrid name property ===
    def name(self) -> str:
        return "fusion_hybrid"

=== yolo_hybrid name property ===
    def name(self) -> str:
        return "yolo_hybrid"

=== motion name property ===
    def name(self) -> str:
        return "motion"
```

48. assistant (2026-07-07T15:15:14.118Z)

The frozenset strings exactly match the registered `name` identifiers ? no typo. Now the yolo question: does `yolo_hybrid` need weights baked on the control-plane, and is it user-selectable in the hosted build? Let me check the person detector's weight source and whether yolo is a serving option.

49. user (2026-07-07T15:15:18.725Z)

```
=== person_detector weight/model source ===
14:  - `lazy_load_model`               ? load a permissively-licensed torchvision detector
22:detector weights load only on first detection (`lazy_load_model`) ? so importing the
segmenter
26:Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT (via the
28:(ADR-005; ultralytics' AGPL YOLO was dropped).
41:# class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
46:# `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
65:        # Dynamic torchvision module (None until lazy_load_model); typed Any so the
70:    def lazy_load_model(self) -> None:
75:        # ships no torch ? P2e) and so the test suite (which pre-sets self.model) never pulls
weights.
100:        # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
101:        self.model = builder(weights="DEFAULT")
140:        This replaces the old ultralytics ``model.track(...)`` call, which bundled
329:        self.lazy_load_model()
384:        self.lazy_load_model()
```

```
=== is yolo_hybrid a serving/selectable option? search config + capabilities + defaults ===
backend/config/models.py:169:    """Config for the multi-signal `fusion_hybrid` segmenter
(MULTI_SIGNAL_FUSION_PLAN P2).
backend/config/models.py:203:    default_segmenter: str = "yolo_hybrid"
backend/config/models.py:285:    # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the
Phase-3 AI handoff entirely and
Binary file backend/config/__pycache__/models.cpython-313.pyc matches
```

50. assistant (2026-07-07T15:15:41.489Z)

Key finding: `yolo_hybrid` uses **torchvision** COCO weights via `weights="DEFAULT"` (auto-download/cache), NOT the WASB/TrackNet trajectory weights the control-plane lacks ? so excluding it is correct, and the fix leaves its behavior unchanged (no regression). Now let me verify `_local_detector_weights_present` actually mirrors the runners' weight resolution (env var name, config field) ? a mismatch would make the guardrail misfire.

51. user (2026-07-07T15:15:45.792Z)

```
=== WASB runner weight resolution (env + config) ===
backend/pipeline/detectors/native_wasb_runner.py:17:    (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` /
``WASB_PYTHON`` / ``WASB_SCRATCH``), then
backend/pipeline/detectors/native_wasb_runner.py:87:    weights_path: str = ""  # REQUIRED (env
WASB_WEIGHTS or indexing.wasb.weights_path)
backend/pipeline/detectors/native_wasb_runner.py:113:        env = os.environ.get
backend/pipeline/detectors/native_wasb_runner.py:120:
weights_path=env("WASB_WEIGHTS") or w.weights_path or "",
backend/pipeline/detectors/native_wasb_runner.py:182:        return
os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
backend/pipeline/detectors/native_wasb_runner.py:202:        if not self.cfg.weights_path:
backend/pipeline/detectors/native_wasb_runner.py:204:                "WASB weights path is empty
? set WASB_WEIGHTS (or indexing.wasb.weights_path)."
backend/pipeline/detectors/native_wasb_runner.py:206:        weights =
os.path.expanduser(self.cfg.weights_path)
backend/pipeline/detectors/native_wasb_runner.py:298:        weights =
os.path.expanduser(self.cfg.weights_path)
backend/pipeline/detectors/wasb_runner.py:45:    weights_path: str = (
backend/pipeline/detectors/wasb_runner.py:46:        "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
backend/pipeline/detectors/wasb_runner.py:71:            weights_path=w.weights_path,
backend/pipeline/detectors/wasb_runner.py:183:            f"--weights
{wsl_tilde_quote(self.cfg.weights_path)} "

=== TrackNet runner weight resolution ===
43:    weights_path: str = ""  # WSL path to the .h5/.keras weights (REQUIRED)
61:            weights_path=tn.weights_path,
142:        if not self.cfg.weights_path:
144:                "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
197:            f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
```

52. assistant (2026-07-07T15:16:24.759Z)

The guardrail's resolution matches the native runners exactly (env `WASB_WEIGHTS` ? `indexing.wasb.weights_path`; tracknet via `indexing.tracknet.weights_path`; `os.path.expanduser`). Now let me verify the fusion `substrate` branch is correct ? is `indexing.fusion.substrate` the real field name and default? A misnamed field would make the guardrail check the wrong weights file.

53. user (2026-07-07T15:16:28.467Z)

```
=== FusionConfig + substrate field ===
18:class ValidationRelevanceConfig(BaseModel):
24:class ValidationConfig(BaseModel):
46:class TrackNetConfig(BaseModel):
51:    weights_path: str = ""
```

```
68:class WasbIndexConfig(BaseModel):
69:    """Typed config for the live WASB shuttle substrate (indexing.wasb).
80:    weights_path: str = (
168:class FusionConfig(BaseModel):
179:    # Which trajectory substrate seeds the windows (shuttle stays primary).
180:    substrate: Literal["wasb", "tracknet"] = "wasb"
182:    # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
202:class IndexingConfig(BaseModel):
293:    tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
294:    wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
295:    fusion: FusionConfig = Field(default_factory=FusionConfig)
324:class OutputConfig(BaseModel):
332:class WatermarkConfig(BaseModel):
354:class StitchingConfig(BaseModel):
370:class LoggingConfig(BaseModel):
380:class SecurityConfig(BaseModel):
413:class StorageConfig(BaseModel):
454:class DeploymentConfig(BaseModel):
486:class BillingConfig(BaseModel):
507:class FlywheelConfig(BaseModel):
529:class AppConfig(BaseModel):
```

54. assistant (2026-07-07T15:16:41.143Z)

`FusionConfig.substrate: Literal["wasb","tracknet"] = "wasb"` (line 180) ? field name and default match the guardrail exactly. Let me read the WASB `weights_path` default and confirm `fusion_hybrid` actually switches runner by substrate.

55. user (2026-07-07T15:16:43.940Z)

```
=== WasbIndexConfig.weights_path default (models.py 68-90) ===
class WasbIndexConfig(BaseModel):
    """Typed config for the live WASB shuttle substrate (indexing.wasb).

    Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
    these knobs actually take effect ? previously this whole block was silently dropped
    because nested config sections defaulted to extra='ignore'.
    """

    wsl_distro: str = "Ubuntu"
    repo_dir: str = "~/models/WASB-SBDT"
    conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
    conda_env: str = "wasb"
    weights_path: str = (
        "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
    )
    sport: str = "badminton"
    wsl_stage_dir: str = "~/clips"
    timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
    velocity_threshold: float = 0.02
    # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner. ---
    # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
    # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4 CPU
fallback
    # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-CPU
downgrade).

=== fusion_hybrid uses substrate to pick runner? ===
3:Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4:TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
22:Defaults reproduce the substrate exactly: net_crossings/person features are persisted but
24:and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the default
35:from backend.pipeline.detectors.tracknet_runner import TrackNetConfig, TrackNetRunner
36:from backend.pipeline.detectors.wasb_runner import WasbConfig, WasbRunner
63:            "Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate rallies,
```

```
"
68:    def _build_runner(
73:        substrate = idx_cfg.fusion.substrate
74:        if substrate == "tracknet":
75:            cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
76:            return TrackNetRunner(cfg), {
77:                "weights_path": cfg.weights_path,
79:                "fusion_substrate": "tracknet",
81:        wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
82:        return WasbRunner(wcfg), {
83:            "weights_path": wcfg.weights_path,
84:            "fusion_substrate": "wasb",
90:        # Native (no-WSL) path supports the WASB substrate only ? there is no native TrackNet
91:        # runner yet. Fusion's default substrate is WASB, so this covers the common case.
```

56. assistant (2026-07-07T15:18:08.582Z)

The guardrail's fusion branch exactly mirrors `fusion_hybrid._build_runner` (substrate?runner?weights_path), and all config field names/defaults match. The backend logic is sound. Now the highest-value SPA check: in the new test, `typeUri` runs while the URI input is **disabled** (cloudBlocked=true before the ack click). If `fireEvent.change` on a disabled input doesn't set state, the test would submit an empty URI and fail. Let me run the SPA test to verify empirically.

57. user (2026-07-07T15:18:11.030Z)

```
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "typecheck": "tsc --noEmit",
    "test": "vitest run",
    "test:coverage": "vitest run --coverage",
    "e2e": "playwright test -c e2e/playwright.config.mjs",
    "e2e:ci": "vite build && node e2e/contract-smoke.mjs && playwright test -c
e2e/playwright.config.mjs",
    "screencast": "vite build && playwright test -c e2e/playwright.config.mjs",
    "screencast:mp4": "node e2e/to-mp4.mjs"
  },
  "dependencies": {
    "@fontsource-variable/noto-sans-devanagari": "^5.2.8",
    "@fontsource-variable/noto-sans-kannada": "^5.2.9",
    "i18next": "^26.3.1",
    "i18next-browser-languagedetector": "^8.2.1",
    "i18next-icu": "^2.4.3",
    "react": "^19.0.0",
    "react-dom": "^19.0.0",
    "react-i18next": "^17.0.8"
```

58. assistant (2026-07-07T15:18:15.359Z)

React 19. Let me run the IngestPanel test suite (on the fix branch) and the typecheck in parallel ? this verifies both the new test passes and no existing test regresses.

59. user (2026-07-07T15:18:24.767Z)

```
=== branch ===
fix/hosted-compute-picker-defaults-cloud
=== run IngestPanel tests ===
stderr | src/components/IngestPanel.test.tsx > IngestPanel >
names the video on the retry screen when a single-shot upload fails (#161)
{"ts":"2026-07-
07T15:18:24.444Z","level":"error","event":"upload.failed","cid":"a2dd23a1-e917-4c49-99f3-
5093d3131767","filename":"Quarterfinal.mp4","error":"ApiError: Network error calling POST
```

```
/api/upload/init"}

stdout | src/components/IngestPanel.test.tsx > IngestPanel >
names the video on the retry screen when a URI submission fails (#161)
{"ts":"2026-07-07T15:18:24.457Z","level":"info","event":"api.request","cid":"429873ff-
56c3-4323-80e5-fab06ea10505","method":"GET","url":"/api/capabilities"}
{"ts":"2026-07-07T15:18:24.463Z","level":"info","event":"ingest.submit_start","cid":"c743344f-
e40b-426b-bab6-1c3644696302","uri":"gs://bucket/SemiFinal.mp4","display_name":"SemiFinal.mp4"}
{"ts":"2026-07-07T15:18:24.463Z","level":"info","event":"api.request","cid":"c743344f-e40b-426b-
bab6-1c3644696302","method":"POST","url":"/api/process"}

stdout | src/components/IngestPanel.test.tsx > IngestPanel >
names the video on the retry screen when a URI submission fails (#161)
{"ts":"2026-07-
07T15:18:24.466Z","level":"info","event":"api.response","cid":"429873ff-56c3-4323-80e5-
fab06ea10505","method":"GET","url":"/api/capabilities","status":200}

stderr | src/components/IngestPanel.test.tsx > IngestPanel >
names the video on the retry screen when a URI submission fails (#161)
{"ts":"2026-07-
07T15:18:24.466Z","level":"error","event":"api.http_error","cid":"c743344f-e40b-426b-bab6-
1c3644696302","method":"POST","url":"/api/process","status":400,"body":"{\"detail\":\"unsupporte
d storage URI scheme: ftp://\"}"}

stderr | src/components/IngestPanel.test.tsx > IngestPanel >
names the video on the retry screen when a URI submission fails (#161)
{"ts":"2026-07-
07T15:18:24.466Z","level":"error","event":"ingest.submit_error","cid":"c743344f-e40b-426b-bab6-
1c3644696302","uri":"gs://bucket/SemiFinal.mp4","code":"engine","detail":"unsupported storage
URI scheme: ftp://"}

stdout | src/components/IngestPanel.test.tsx > IngestPanel >
still names the video on the retry screen after a reload, once the resumed job fails
(#161)
{"ts":"2026-07-
07T15:18:24.475Z","level":"info","event":"ingest.resume","cid":"c1","uri":"gs://b/reload.mp4","j
ob_id":"jR"}
{"ts":"2026-07-07T15:18:24.475Z","level":"info","event":"api.request","cid":"a84f30bb-4d69-47c3-
a893-b6d141eb6c60","method":"GET","url":"/api/capabilities"}

stdout | src/components/IngestPanel.test.tsx > IngestPanel >
still names the video on the retry screen after a reload, once the resumed job fails
(#161)
{"ts":"2026-07-
07T15:18:24.478Z","level":"info","event":"api.response","cid":"a84f30bb-4d69-47c3-a893-
b6d141eb6c60","method":"GET","url":"/api/capabilities","status":200}

stdout | src/components/IngestPanel.test.tsx > IngestPanel >
still names the video on the retry screen after a reload, once the resumed job fails
(#161)
{"ts":"2026-07-
07T15:18:24.479Z","level":"info","event":"api.request","cid":"c1","method":"GET","url":"/api/job
s/jR"}

stdout | src/components/IngestPanel.test.tsx > IngestPanel >
still names the video on the retry screen after a reload, once the resumed job fails
(#161)
{"ts":"2026-07-
07T15:18:24.479Z","level":"info","event":"api.response","cid":"c1","method":"GET","url":"/api/jo
bs/jR","status":200}

stderr | src/components/IngestPanel.test.tsx > IngestPanel >
still names the video on the retry screen after a reload, once the resumed job fails
(#161)
{"ts":"2026-07-
```

```
07T15:18:24.480Z","level":"error","event":"ingest.job_failed","cid":"c1","uri":"gs://b/reload.mp
4","job_id":"jR","code":"engine","detail":"worker crashed"}

 ? src/components/IngestPanel.test.tsx (23 tests)
903ms

 Test Files  1 passed (1)
      Tests  23 passed (23)
   Start at  20:48:20
   Duration  4.04s (transform 238ms, setup 18ms, collect 634ms, tests 903ms,
environment 1.84s, prepare 233ms)
```

60. assistant (2026-07-07T15:18:33.812Z)

All 23 IngestPanel tests pass, including the new one ? confirming `fireEvent.change` sets state even on the disabled input, and no existing SPA test regresses. Let me run the typecheck to confirm the `compute_target` read doesn't break the build.

61. user (2026-07-07T15:18:41.479Z)

```
=== typecheck ===
=== exit: 0 ===
```

62. assistant (2026-07-07T15:18:58.591Z)

Typecheck passes (exit 0) ? no TS regression. Now let me verify the backend tests. The fix is on a different branch than the backend working tree (which has sibling WIP), so I'll use a temporary git worktree to avoid clobbering it. First, let me check for an existing venv and how backend tests run.

63. user (2026-07-07T15:19:04.439Z)

```
=== venvs / python ===
=== python? ===
/c/Users/avidu/AppData/Local/Programs/Python/Python313/python
Python 3.13.13
=== pytest config ===
pyproject.toml
pytest.ini
=== can we import backend deps? quick check ===
fastapi+pydantic OK
```

64. assistant (2026-07-07T15:19:11.108Z)

Python 3.13 with fastapi/pydantic available. Let me check the test file's imports and set up a worktree for the fix branch to run the backend tests safely.

65. user (2026-07-07T15:19:15.447Z)

```
=== test file imports (first 30 lines) ===
"""API cloud-serving dispatch (COMPUTE_DECOUPLED_SERVING M6 ? the API path).

Proves the /api/process job path can offload DETECT to a Cloud Run GPU Job and ingest the rally
intervals back into the API DB, so /api/query + /api/compile (local stitch) are unchanged ? all
GPU-free + cloud-free via a FAKE CloudRunJobClient that writes report.json + intervals.csv + the
done-marker into a local "bucket". Cardinal invariant: DEFAULT-OFF ? only
compute_target=cloud_run
dispatches; every other target runs the in-process segmenter byte-identically.
"""

import csv
import json
import os
```

```
import pytest

pytest.importorskip("httpx")
from fastapi.testclient import TestClient

import backend.compute as compute_pkg
import backend.main as m
from backend.compute.cloud_run import CloudRunJobClient
from backend.config.models import AppConfig
from backend.infrastructure.database import Database
from backend.infrastructure.execution_policy import ExecutionPolicy
from backend.pipeline.validators.base import ValidationResult

_FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)


def _pass():

=== pytest available? ===
pytest 9.0.3
```

66. assistant (2026-07-07T15:19:26.959Z)

pytest 9.0.3 available; tests use fakes (no torch needed). Let me create a worktree for the fix branch and run the test file.

67. user (2026-07-07T15:19:31.821Z)

```
=== creating worktree for fix branch (read-only verify) ===
Preparing worktree (detached HEAD 4565c74)
HEAD is now at 4565c74 fix(serving): fail fast when compute=local forces weightless in-process
detection
=== confirm the guardrail is present in the worktree ===
3
```

68. assistant (2026-07-07T15:19:37.110Z)

Worktree created at the fix commit with the guardrail present. Now let me run the test file from it.

69. user (2026-07-07T15:20:18.116Z)

```
=== run test_api_cloud_dispatch.py (all incl. 4 new guardrail tests) ===
................................                                     [100%]
============================== warnings summary ==============================
tests/test_api_cloud_dispatch.py::test_full_api_process_to_query_via_cloud
tests/test_api_cloud_dispatch.py::test_full_api_process_to_query_via_cloud
tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
  C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
    warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
34 passed, 8 warnings in 36.33s
```

70. assistant (2026-07-07T15:20:34.272Z)

All 34 tests pass (30 existing + 4 new); the warnings are just Google auth noise. Let me check
the rest of the backend test suite for any OTHER test that does `compute="local"` with a
weights-backed segmenter (which could regress), then clean up the worktree.

71. user (2026-07-07T15:20:39.508Z)

```
=== any OTHER test file combining compute=local with a weights-backed segmenter? ===
tests/test_restart_recovery.py
tests/__pycache__/test_api_cloud_dispatch.cpython-313-pytest-9.0.3.pyc
--- occurrences with wasb/tracknet/fusion near compute=local across tests ---
tests/test_restart_recovery.py:392:        "jlocal", SOURCE_URI, video_id=MD5, compute="local"
tests/test_restart_recovery.py:405:        "jlocal2", SOURCE_URI, video_id=MD5, compute="local"
=== done grep ===
```

72. assistant (2026-07-07T15:20:43.217Z)

`tests/test_restart_recovery.py` also uses `compute="local"`. Let me check what segmenter it
uses and whether the guardrail could affect it.

73. user (2026-07-07T15:20:47.549Z)

```
=== context around compute=local in test_restart_recovery.py (370-420) ===
    fake = _FakeBucket()  # an older SUCCESSFUL report is sitting in the bucket
    monkeypatch.setattr(storage_pkg, "get_storage", lambda *a, **k: fake)
    monkeypatch.setattr(job_worker, "_RESUME_POLL_EVERY_SEC", 0.0)

    job_worker._resume_one_remote_job(cfg, db, db.get_job(job_id))

    job = db.get_job(job_id)
    assert job["status"] == "failed"
    assert "force" in (job["error"] or "")
    # The stale cached result was NOT ingested as this job's outcome.
    assert job["result"] in (None, {})
```

--- only PROVEN cloud-dispatched rows resume (#494 review round 2) ----------

```
def test_recover_fails_explicit_local_running_jobs(tmp_path):
    """The round-2 repro: compute='local' forces the in-process segmenter ? no remote
    worker exists, so a restart must fail the row, never complete it from a same-md5
    bucket report left by an OLDER run."""
    db = Database(str(tmp_path / "t.db"))
    claimed = db.claim_or_create_process_job(
        "jlocal", SOURCE_URI, video_id=MD5, compute="local"
    )
    assert claimed and claimed["created"]
    db.mark_job_running("jlocal")
    requeued, failed = db.recover_stale_jobs()
    assert (requeued, failed) == (0, 1)
    assert db.get_job("jlocal")["status"] == "failed"
    assert db.interrupted_remote_process_jobs() == []


def test_resume_one_refuses_an_explicit_local_job_defense_in_depth(tmp_path, monkeypatch):
    db = Database(str(tmp_path / "t.db"))
    claimed = db.claim_or_create_process_job(
        "jlocal2", SOURCE_URI, video_id=MD5, compute="local"
    )
    assert claimed and claimed["created"]
    db.mark_job_running("jlocal2")
    fake = _FakeBucket()  # an older SUCCESSFUL report is sitting in the bucket
    monkeypatch.setattr(storage_pkg, "get_storage", lambda *a, **k: fake)
    monkeypatch.setattr(job_worker, "_RESUME_POLL_EVERY_SEC", 0.0)
```

```
    job_worker._resume_one_remote_job(_gcs_cfg(tmp_path), db, db.get_job("jlocal2"))
    job = db.get_job("jlocal2")
    assert job["status"] == "failed"
    assert job["result"] in (None, {})


def test_null_compute_resumes_only_when_the_server_default_is_cloud(tmp_path, monkeypatch):
    """compute IS NULL means "server default": remote only when the default target itself is
    cloud_run. On a default-local server that merely HAS cloud-override capability, a NULL
=== does it set a cloud server / weights-backed segmenter? grep the file ===
152:    cfg.deployment.compute_target = "cloud_run"
387:    """The round-2 repro: compute='local' forces the in-process segmenter ? no remote
420:    cloud_run. On a default-local server that merely HAS cloud-override capability, a NULL
430:    cfg = _gcs_cfg(tmp_path)  # deployment.compute_target stays "" (default-local)
454:    cfg.deployment.compute_target = "cloud_run"  # the server default IS cloud
```

74. assistant (2026-07-07T15:20:57.110Z)

These `test_restart_recovery.py` tests exercise the DB job-recovery layer (`recover_stale_jobs`, `_resume_one_remote_job`), not `_maybe_dispatch_remote`/`_execute_processing` ? a separate code path the guardrail doesn't touch. Let me run it to confirm, then clean up.

75. user (2026-07-07T15:21:07.868Z)

```
=== run restart recovery (unaffected path) ===
.......................                                                  [100%]
24 passed in 4.57s
```

76. assistant (2026-07-07T15:21:54.682Z)

All 24 restart-recovery tests pass ? that layer is untouched. Let me clean up the temporary worktree.

77. user (2026-07-07T15:21:58.535Z)

```
=== worktrees now ===
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
3be0935 [fix/cloud-run-exec-status-fastfail]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/cfg-wt        20b2d5a [claude/wasb-
config-validation]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/compile-wt    b09a1d4 [claude/compile-
reliability]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-pre-wt  355dad6 (detached HEAD)
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-wt      5de1ef2 (detached HEAD)
C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval
8586c34 [fix/worker-skip-revalidation-on-dispatch]
=== main working tree branch unchanged ===
fix/cloud-run-exec-status-fastfail
```

78. assistant (2026-07-07T15:22:48.722Z)

Verification complete. The temporary worktree is removed cleanly; the main tree's branch churn (`perf/validation-shared-decode-pass` ? `fix/cloud-run-exec-status-fastfail`) is a concurrent sibling session's doing, not mine ? my checks ran against commit `4565c74` directly via the detached worktree, so they're unaffected.

Summary of my dimension (conventions-and-regressions) ? no real defects found. What I verified:

Regressions ? none:
- All existing `compute="local"` tests in `tests/test_api_cloud_dispatch.py` use

`segmenter="motion"` (lines 489, 659, 866, 1004); the only `compute="local"`+`wasb_hybrid` tests are the 4 new ones. The guardrail is scoped to `_WEIGHTS_BACKED_SEGMENTERS`, so motion is untouched. Ran the file: 34/34 pass.
- `test_local_override_survives_api_boundary` (the pre-existing cloud+local boundary test) uses `motion` ? unaffected.
- `tests/test_restart_recovery.py` `compute="local"` cases exercise the DB job-recovery layer (`recover_stale_jobs`/`_resume_one_remote_job`), not `_maybe_dispatch_remote` ? 24/24 pass.
- SPA: the default-to-cloud switch is gated on `compute_target === "cloud_run"`, which no existing IngestPanel test sets (they use `cloud_available:true` without `compute_target`, or `false`), so the `compute=local` default assertion at line 185 and the "gates cloud on a cost ack" test still hold. Ran the file: 23/23 pass. `tsc --noEmit` clean (`compute_target?` is declared in `types.ts:109`, emitted by `main.py:401`).

Convention fit ? consistent:
- `_WEIGHTS_BACKED_SEGMENTERS = {wasb_hybrid, tracknet_hybrid, fusion_hybrid}` exactly matches the registered `name` identities; `trajectory_hybrid` is the deliberately-unregistered base (`trajectory_hybrid.py:18`); `yolo_hybrid` uses torchvision auto-download COCO weights (`person_detector.py:100-101`), not the GCSFuse-mounted trajectory weights, so its exclusion is correct and its behavior is unchanged.
- `_local_detector_weights_present` mirrors the native runner resolution exactly: `WASB_WEIGHTS` env ? `indexing.wasb.weights_path` (`native_wasb_runner.py:120`), TrackNet via `indexing.tracknet.weights_path` (`tracknet_runner.py:61`), fusion by `indexing.fusion.substrate` (`fusion_hybrid.py:73`), and `os.path.expanduser` matches `native_wasb_runner.py:206`. `os` is already imported (`orchestration.py:25`).
- `_failed(msg, False)` message tone matches the sibling `choice=="cloud"` reject (lowercase-first, actionable, suggests the alternative); provenance `path=not_run, requested=local, dispatched=False` is preserved by the `finally` `setdefault` at line 906.

SPA cost-ack interaction ? correct: on a hosted `cloud_run` build, `compute` defaults to `"cloud"` ? `cloudBlocked=true` disables both inputs until the ack, so no paid job dispatches without consent; `sendCompute="cloud"`; the ack checkbox itself is never disabled (user can't get stuck); a deliberate switch to `local` falls to the paired backend fast-fail (actionable, not the cryptic trap). No new i18n keys/testids. The pre-caps-resolve race (submit before `/api/capabilities` returns ? `compute` omitted ? server default) is pre-existing and not worsened by this change.

79. user (2026-07-07T15:22:48.793Z)

Structured output provided successfully